HKDSE ICT • SCHOOL-BASED ASSESSMENT

Hong Kong Diploma of Secondary Education
Information and Communication Technology
School-based Assessment

Mastermind
Game System

Candidate name	CHAN, YU SHING
Candidate number	Student ID s202101127
School	CCC Ming Kei College
Submission	1 September 2026 • CLI-centred system report

DESIGN • IMPLEMENTATION • TESTING • EVALUATION

Student Declaration and Verification Scope

This report documents the Mastermind Game System implemented by CHAN, YU SHING (Student ID s202101127) of CCC Ming Kei College in the accompanying repository. The student should confirm authorship, cite all assistance required by school policy, and retain the tested repository baseline.

Technical statements are grounded in source code at commit 67b4cc9be06961483661bf3219626a4ea2e6014e and verification executed on 1 September 2026. This baseline includes the serialized concurrent-duel repair at commit 13fa42f0d9703f945881610a9a2d89ac7d31446f. The report does not claim that every reconstructed debugging scenario corresponds to an individually preserved historical commit; Section 7 identifies the final safeguards and their regression evidence.

Student signature	_____
Teacher signature	_____
Date	_____

Executive Summary

This SBA presents a command-line Mastermind system built in Python 3.13 around a shared canonical rule engine. Players can choose four presets or Custom mode, play against a secure computer Code Maker or a human in pass-and-play, receive duplicate-safe feedback, review local scores, and export validated CSV data. The implementation separates interaction, deterministic rules, and persistence through typed immutable models and dependency injection.

> *Verification result: **117 Python tests passed** and **36 browser tests passed** with 8 intentional browser skips; focused CLI/core tests achieved **92% measured coverage**, with clean Ruff and mypy checks.*

Table of Contents

The entries below are linked to their sections. Page numbers are refreshed from the rendered report during the build process.

List of Figures and Tables

Figures

Principal tables

1: Problem Definition

Mastermind is a deductive code-breaking game. A Code Maker selects a hidden ordered sequence of colour identifiers and a Code Breaker submits guesses. After each valid guess, the system returns black feedback pegs for correct colours in correct positions and white feedback pegs for correct colours in wrong positions. The computational challenge is not simply comparing two lists: duplicate colours must be counted once, invalid data must never alter the game, terminal states must stop accepting guesses, and the final result must be saved reliably.

The project delivers this solution primarily through a Python command-line interface (CLI). The terminal application is intentionally understandable as an SBA program, but it is not a disposable prototype. It imports the same `mastermind_core` package as the production web API. This makes the CLI a compact demonstration of decomposition, selection, iteration, validation, structured data, file handling, testing, and defensive programming.

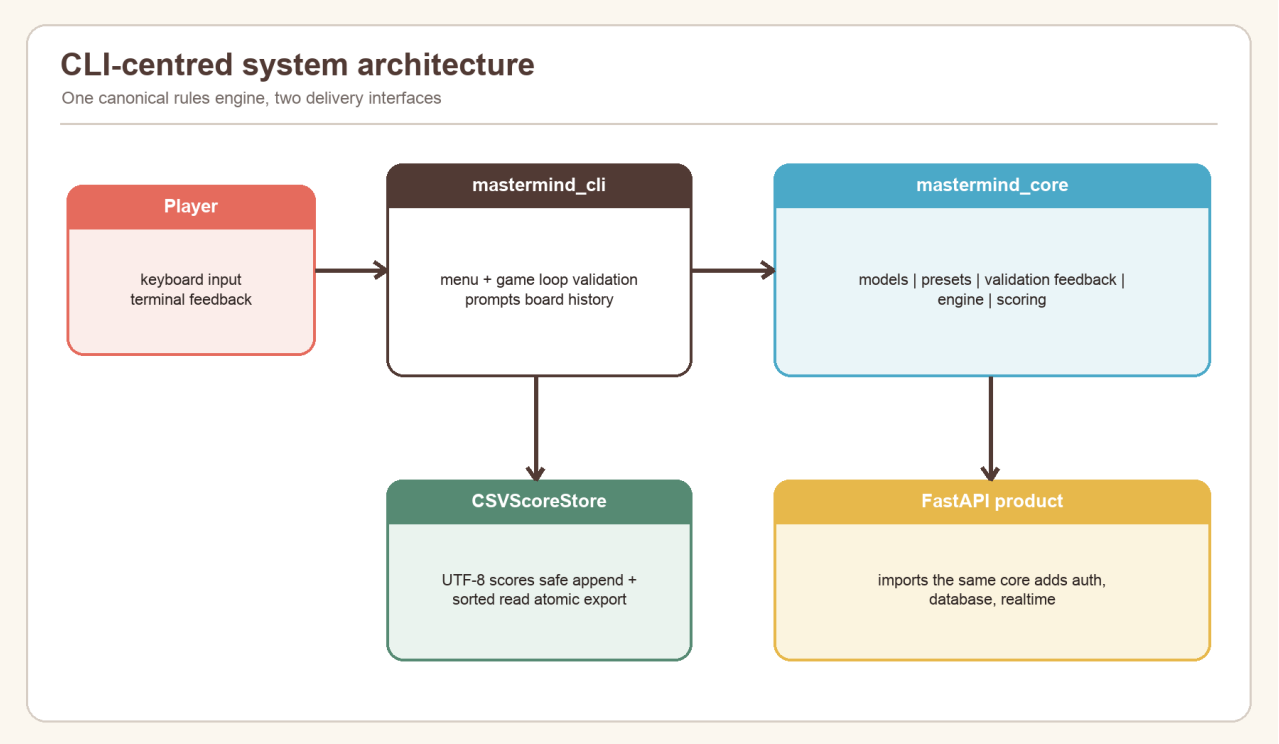

Figure 1. CLI-centred system architecture and shared canonical engine.

1.1: Background

The physical Mastermind board uses coloured pegs and small feedback pegs. Converting it into a CLI requires a stable text representation. The program uses the identifiers `R`, `B`, `G`, `Y`, `W`, `K`, `O`, `P`, `C`, and `M`. A user may enter a compact code such as `RBGY` or a separated form such as `R B G Y`, `R,B,G,Y`, or `R-B-G-Y`; all valid forms become the same immutable tuple.

The first implementation boundary is between interaction and rules. `mastermind_cli.app` owns prompts, menus, and display. `mastermind_core` owns configuration invariants, code validation, feedback, state transitions, secret generation, and scoring. `mastermind_cli.storage` owns local CSV persistence. This separation reduces coupling: a change in terminal wording does not change the scoring formula, while a correction to feedback automatically benefits both the CLI and API.

The project is also designed for realistic failure. A user can mistype a menu option, submit a malformed guess, interrupt the program, provide a damaged CSV file, or export to an unavailable location. The CLI converts these conditions into clear messages and returns to a safe state rather than presenting an unhandled traceback.

1.2: Objective

The primary objective is to create a reliable, testable Mastermind system that can be operated entirely from a terminal. The completed CLI must:

- present a five-option menu for starting a game, viewing scores, reading rules, exporting scores, and exiting;
- support Easy, Normal, Hard, Expert, and Custom configurations;
- support both computer and human Code Makers, hiding human secret input;
- normalize readable input formats and reject invalid guesses without consuming an attempt;
- calculate duplicate-safe black and white feedback;
- display the complete attempt history and attempts remaining after every accepted guess;
- handle win, loss, abandonment, end-of-input, and keyboard interruption safely;
- calculate a reproducible, versioned score;
- create, append, validate, sort, and export UTF-8 CSV records; and
- reuse the canonical rule engine shared by the wider Cipherboard product.

Success is judged by observable behaviour and repeatable verification, not by the amount of code written. Each objective is traced to implementation and tests later in this report.

1.3: Minimum Expectations

The minimum viable system is a menu-driven Python program that generates a legal secret, accepts guesses, returns correct feedback, ends at the right time, and stores results. The delivered system exceeds this baseline through custom constraints, pass-and-play, hidden secret entry, immutable domain objects, versioned scoring, formula-injection protection, atomic exports, malformed-row recovery, a complete automated test suite, and cross-platform installers.

Requirement	Minimum expectation	Delivered evidence
Gameplay	One working game loop	Presets, Custom, computer/human Code Maker
Validation	Reject wrong length	Empty, separator, palette, duplicate, and boundary checks
Feedback	Black and white counts	Linear duplicate-safe Counter algorithm
Persistence	Save a score	UTF-8 append, sorted read, safe atomic export
Reliability	Normal exit	quit, EOF, interrupt, disk and malformed-file handling
Quality	Manual checks	117 Python and 36 browser tests passing; focused 92% coverage

1.4: System Development Cycle

I followed an iterative system development cycle rather than attempting to write the entire program in one pass. Problem definition established the rules and user needs. Analysis converted the rules into parameters, constraints, inputs, processes, and outputs. Design separated the canonical engine, CLI controller, and storage adapter. Implementation used typed Python modules and frozen dataclasses. Testing progressed from pure functions to CLI integration and file-system behaviour. Evaluation then identified strengths, limitations, and future work.

The cycle was repeated whenever a boundary case appeared. For example, duplicate feedback led to a two-stage matching design; CSV exports led to formula protection and atomic replacement; pass-and-play led to hidden input and a hand-off screen. The Gantt chart in Appendix 9.2 summarises the planned sequence, while Section 7 records concrete debugging cases.

2: System Analysis

System analysis identifies the technologies, interaction model, operating assumptions, and boundaries that shape the solution. The CLI is deliberately lightweight: it does not require a terminal UI framework, database server, or network connection. That keeps the school-facing program inspectable while still applying production-quality design principles.

2.1: Programming Language

Python 3.13.14 is the pinned project runtime. Python suits the task because its sequence operations, dataclasses, enums, standard-library CSV support, `Counter`, and testing ecosystem express the problem directly. Type annotations improve readability and allow mypy to detect mismatched values before execution. The project uses modern syntax such as `StrEnum`, `slots=True`, union types, and `zip(..., strict=True)`.

The language choice also supports safe standard-library components. `secrets.SystemRandom` supplies operating-system randomness for normal secret generation; `getpass` hides a human Code Maker's input; `csv.DictReader` and `DictWriter` preserve an explicit schema; `tempfile.mkstemp` and `os.replace` support atomic export. The program does not use `eval`, shell interpolation, or a custom CSV parser.

Language feature	Application in this system	Benefit
Frozen dataclasses	Game configuration, state, attempts, score	Prevent accidental mutation
Enums	Mode, status, Code Maker, visibility	Prevent spelling-dependent states
Tuples	Palette, secret, guess, attempt history	Stable ordered values
Exceptions	Stable DomainError codes and messages	Separates invalid data from control flow
Protocol	Injectable random source	Secure production behaviour and deterministic tests

2.2: User Interface

The user interface is a conversational terminal menu. This matches the CLI focus and permits complete keyboard operation. The opening menu contains exactly five numbered choices. Prompts state valid ranges, and recovery messages explain the correction needed. During a game, a fixed-width attempt table exposes number, guess, black feedback, white feedback, and attempts remaining.

The terminal interface avoids relying on colour itself. Pegs are represented by stable letters, so the game remains usable in monochrome terminals and by users who cannot distinguish some colours. Human secret input uses `getpass`, and forty blank lines separate the Code Maker hand-off from the Code Breaker view. Although this is not as strong as a separate device, it is appropriate for local pass-and-play.

Figure 2. Menu, invalid-choice recovery, and rules output.

2.3: Integrated Development Environment (IDE)

The program is editor-independent. Development can be completed in Visual Studio Code, PyCharm, or another Python-capable IDE because the repository defines its environment in `pyproject.toml`, `.python-version`, and `uv.lock`. `uv sync --frozen --all-extras` installs the exact development dependencies; `uv run` executes commands inside the project environment.

Useful IDE features include type-aware navigation, test discovery, breakpoint debugging, and integrated terminals. However, the authoritative checks are command-line commands that also run in GitHub Actions: Ruff formatting and linting, mypy strict type checking, pytest, and branch coverage. This prevents a local IDE setting from becoming an undocumented requirement.

3: Problem Analysis

The original example structure uses timetable terminology. To retain every required heading without misrepresenting this project, this section maps each timetable term to its Mastermind equivalent. A timetable row becomes an attempt row; teacher assignment becomes role assignment; subject selection becomes difficulty selection; classroom generation becomes game-session generation; and timetable regeneration becomes replay with a new secret.

3.1: Timetable Composition

In this Mastermind adaptation, “timetable composition” means the composition of one complete game board over time. Each row contains an attempt number, ordered guess, black count, white count, and submission time. Rows are append-only and retain chronological order. The configuration determines the number of columns in the logical code and the maximum number of attempt rows.

The CLI reconstructs the visible board from `GameState.attempts` after every accepted guess. It does not maintain a second display-specific history that could disagree with the engine. This is a useful invariant: the same tuple that is scored and tested is the tuple printed to the player.

3.2: General Parameters

General parameters define the legal game space. The palette contains 5–10 unique identifiers; code length is 3–6; maximum attempts is 1–20; duplicates may be allowed or forbidden; and the Code Maker may be a computer or human. `ranked` and `visibility` exist in the shared model, although the local CLI records ordinary preset games and custom games without a server leaderboard.

Parameter	Type	Valid values	CLI source
Enabled colours	tuple of strings	5–10 unique known IDs	preset or custom prompt
Code length	integer	3–6	preset or custom prompt
Maximum attempts	integer	1–20	preset or custom prompt
Duplicates	Boolean	yes/no	preset or custom prompt
Code Maker	enum	computer/human	preset defaults or custom prompt
Difficulty label	string	easy/normal/hard/ expert/custom	menu selection

3.3: Teacher Parameters

“Teacher parameters” are mapped to role and authority parameters. The Code Maker owns the secret; the Code Breaker owns guesses. When the computer is Code Maker, the engine generates the secret through `SystemRandom`. When a human is Code Maker, the same `validate_code` function checks the hidden entry before the device is handed over.

The separation prevents authority leakage. The Code Breaker never receives the secret through normal output while the game is active. The CSV record intentionally excludes the secret. Only a lost game reveals the code, because no further competitive attempt can occur. A won game does not need to reveal it because the successful final guess already shows the sequence.

3.4: Timetable Illustration

The terminal attempt table is the timetable illustration for this project. A valid guess creates exactly one new row. An invalid guess creates no row and does not reduce `attempts_remaining`. The full history is printed rather than only the newest attempt, so the player can compare patterns and reason deductively.

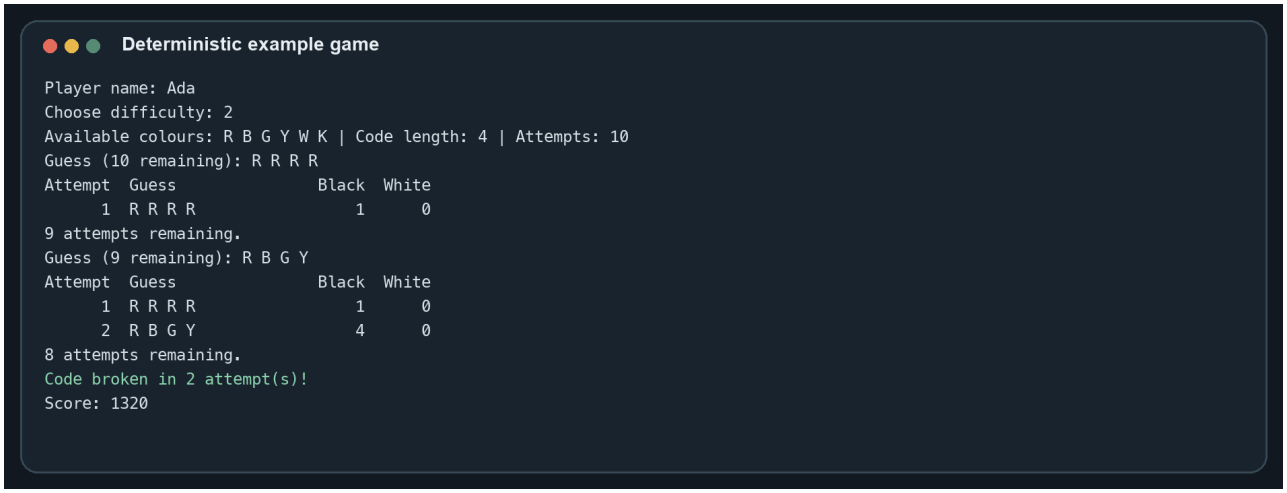

Figure 3. Example attempt history and completed game.

3.5: Constraints

Constraints protect both the rules and data integrity. Configuration constraints are enforced when GameConfig is constructed, so invalid presets cannot exist. Guess constraints are enforced before an AttemptRecord is created. State constraints reject attempts after a terminal result. Persistence constraints require the full CSV header and parse each row defensively.

The most important gameplay constraint is single-use feedback: one secret peg can contribute to at most one black or white peg. Exact matches must be removed before colour-only matches are counted. Other important constraints include no duplicate palette identifiers, enough colours when repetition is forbidden, a stable score version, and no secret in local score data.

3.5.1: Constraint Simplification

Complex rules are simplified into layered checks. normalize_code handles representation; validate_code handles game-specific legality; submit_guess handles state and transition; calculate_feedback handles comparison; and calculate_score handles terminal points. Each function has one clear reason to reject data.

This layering makes proofs and tests smaller. For example, calculate_feedback can assume two equal-length sequences because the engine validates them first, yet it still contains its own length guard for direct callers. The overall behaviour becomes a composition of simple invariants rather than one long function with nested conditions.

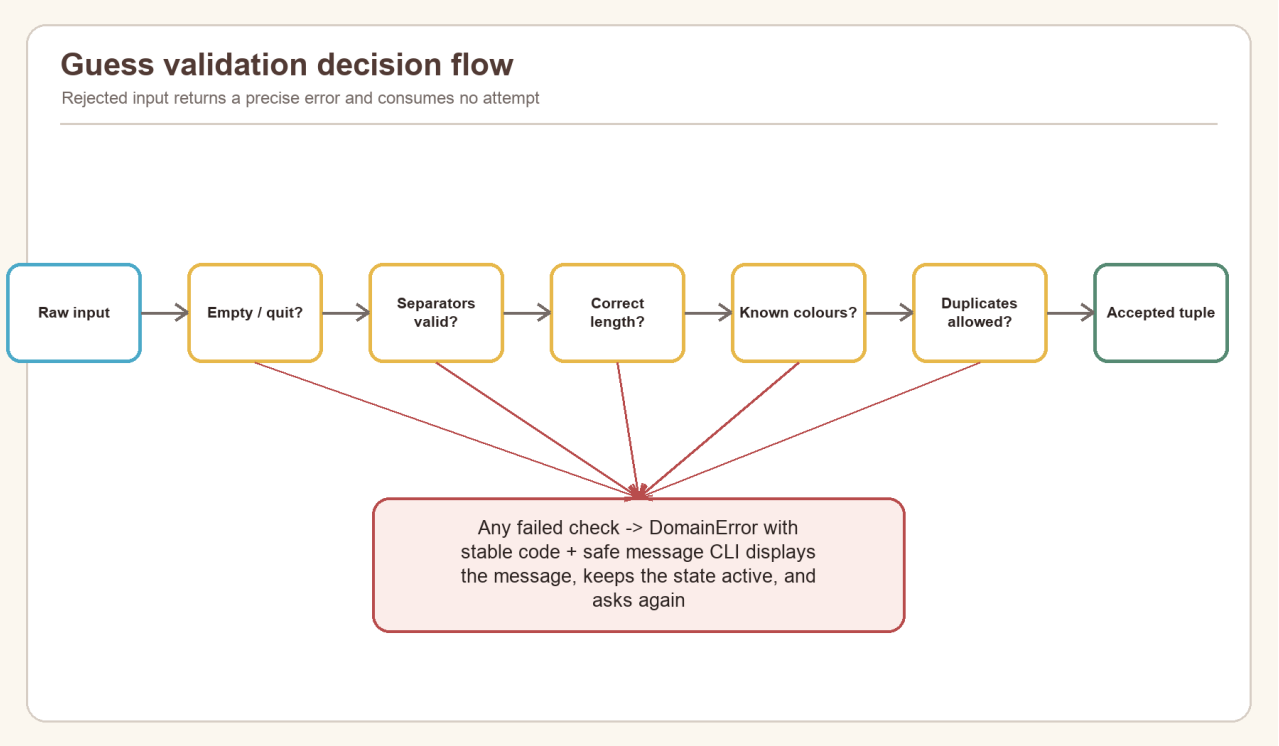

Figure 4. Layered validation flow; rejected guesses consume no attempt.

3.6: IPO Cycle

The Input-Process-Output cycle describes the program independently of implementation details. Inputs include menu choices, player name, configuration values, secret entry, guesses, and export path. Processing includes normalization, constraint checking, secure generation, feedback, state transition, scoring, sorting, and file writing. Outputs include rules, validation messages, attempt history, terminal result, high-score table, and CSV export confirmation.

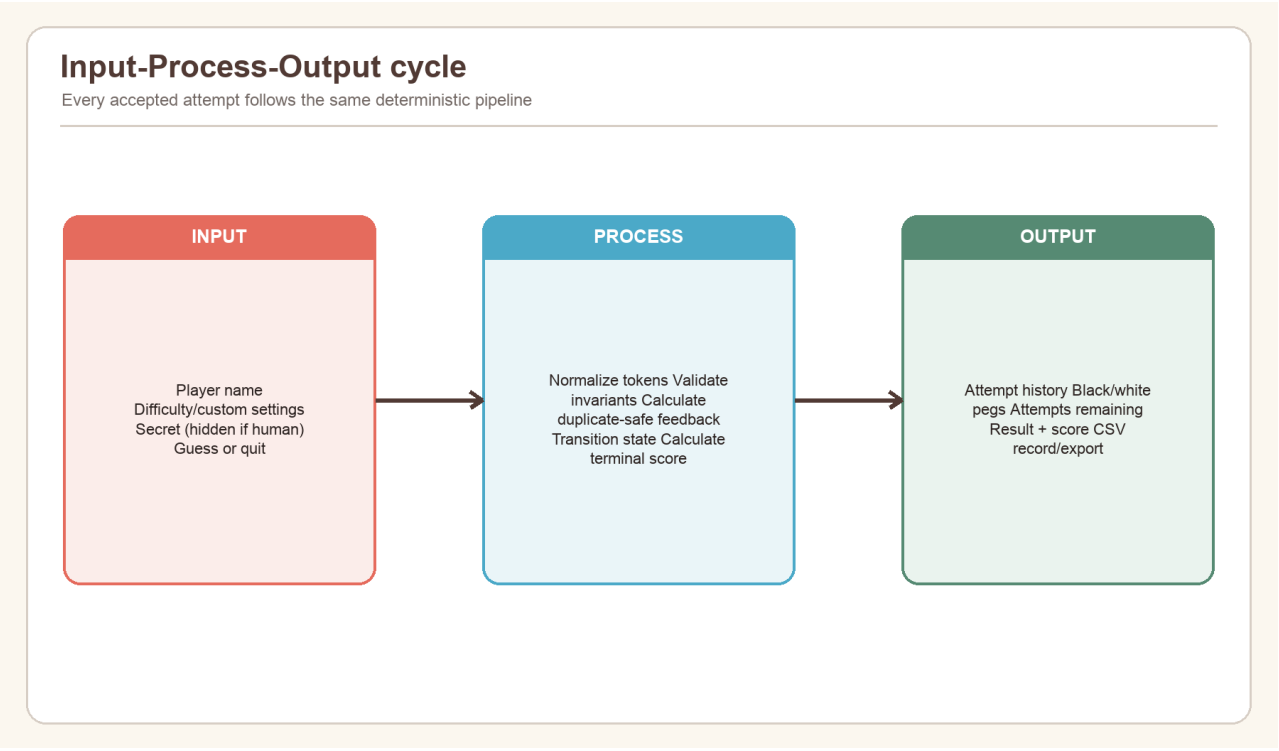

Figure 5. Input-Process-Output cycle for one accepted attempt.

3.6.1: Command Inputs

Command inputs are line-oriented strings. `.strip()` removes accidental surrounding spaces, `.casefold()` supports case-insensitive commands, and numeric prompts convert with `int` inside a retry loop. Guess input accepts

letters without separators or tokens separated by whitespace, commas, or hyphens. A case-insensitive `quit` command abandons the active game.

Input functions are injected into `MastermindCLI`, which is significant for testability. Production uses `input` and `getpass`; tests supply deterministic callables. This avoids patching the entire terminal and makes every prompt path reproducible.

3.6.2: Terminal Outputs

Output is also injected. Production uses `print`; tests append each message to a list. The CLI uses safe user-facing `DomainError.message` values rather than tracebacks or internal object representations. Tabular rows use explicit widths so values remain aligned for normal player names and settings.

Output follows progressive disclosure: the main menu is short, rules are shown on request, game parameters appear once a session begins, history appears after accepted attempts, and a warning appears only when persistence fails. This keeps routine play readable while retaining diagnostic information.

3.6.3: CSV Export

CSV export reads the validated local records, writes them to a temporary file in the destination directory, flushes and synchronizes the file, and then atomically replaces the requested target. If writing fails, the temporary file is removed and the old destination remains intact. Values beginning with spreadsheet formula markers are prefixed with an apostrophe.

The export always writes the canonical 13-column header even when there are no records. This produces a useful empty dataset and simplifies downstream processing. The default path is `mastermind_scores.csv`, but the user may enter another path.

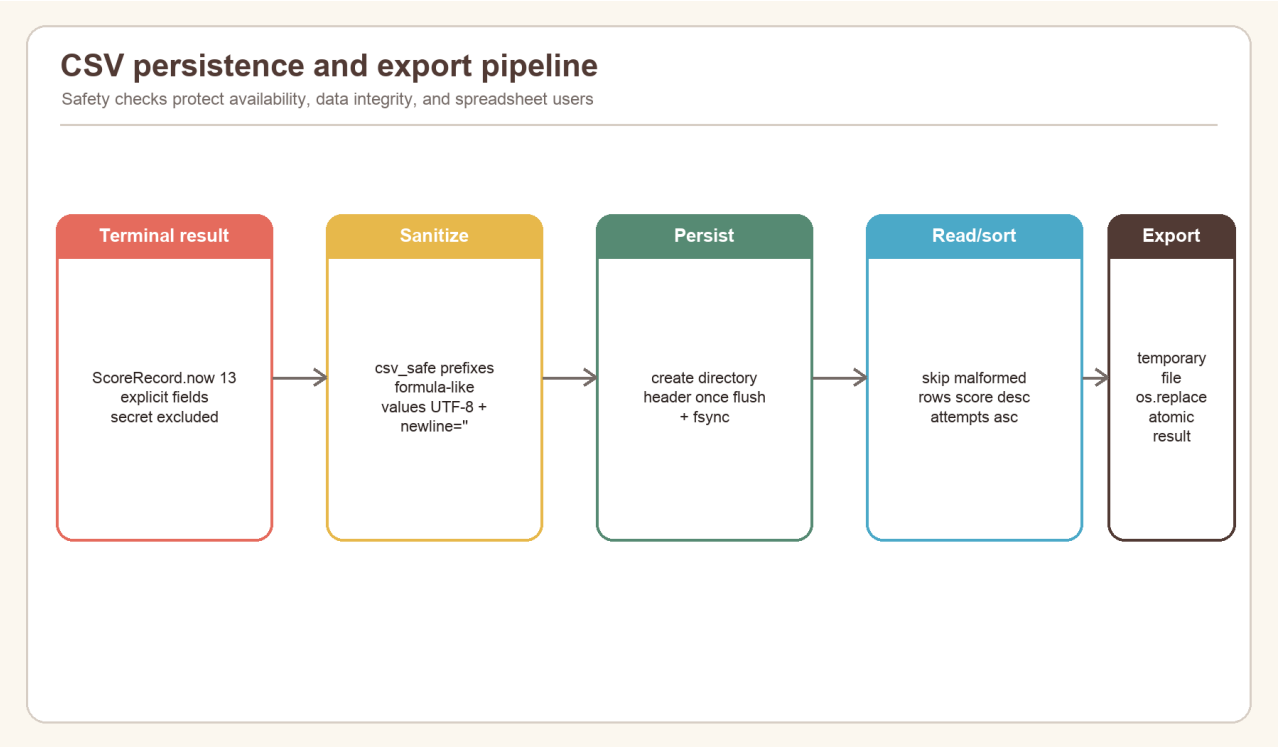

Figure 6. Defensive local persistence and atomic export pipeline.

3.6.4: Report Output

The principal operational report is the local high-score view. Records are sorted by score descending, attempts ascending, then timestamp ascending. The first twenty are printed with rank, player, difficulty, score, and attempt usage. CSV export provides the detailed machine-readable report with configuration and scoring version.

This SBA document is a second report output: it connects requirements, design, source evidence, tests, and evaluation. All screenshots and charts are generated from repository code or measured verification results, so figures can be reproduced rather than manually edited.

3.7: Modularization

The solution is modularized around dependency direction. `mastermind_cli.app` depends on the public exports of `mastermind_core` and on `CSVScoreStore`. The core never imports the CLI or file system. Within the core, `engine.py` coordinates models, validation, feedback, and scoring; it does not duplicate their logic.

Graph analysis of the repository found 1,392 code nodes and 4,574 structural edges across 183 code files. For the CLI-focused subgraph, `engine.py` is the main bridge between immutable models and rule functions. This supports the design decision to keep interaction at the edge and deterministic rules at the centre.

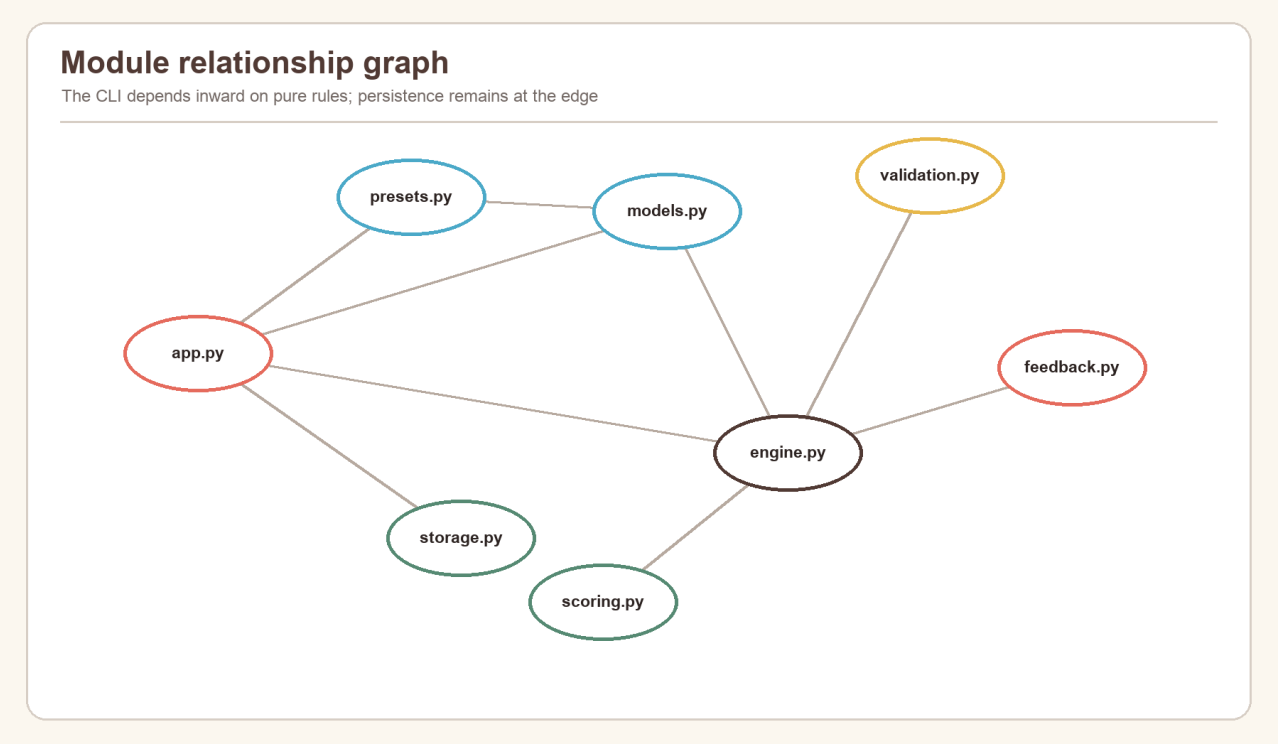

Figure 7. CLI and canonical-core module relationship graph.

4: Program Design

Program design translates the analysis into components, flows, and data contracts before implementation details are considered. The design uses a controller (MastermindCLI), a pure domain package (mastermind_core), and a local repository adapter (CSVScoreStore).

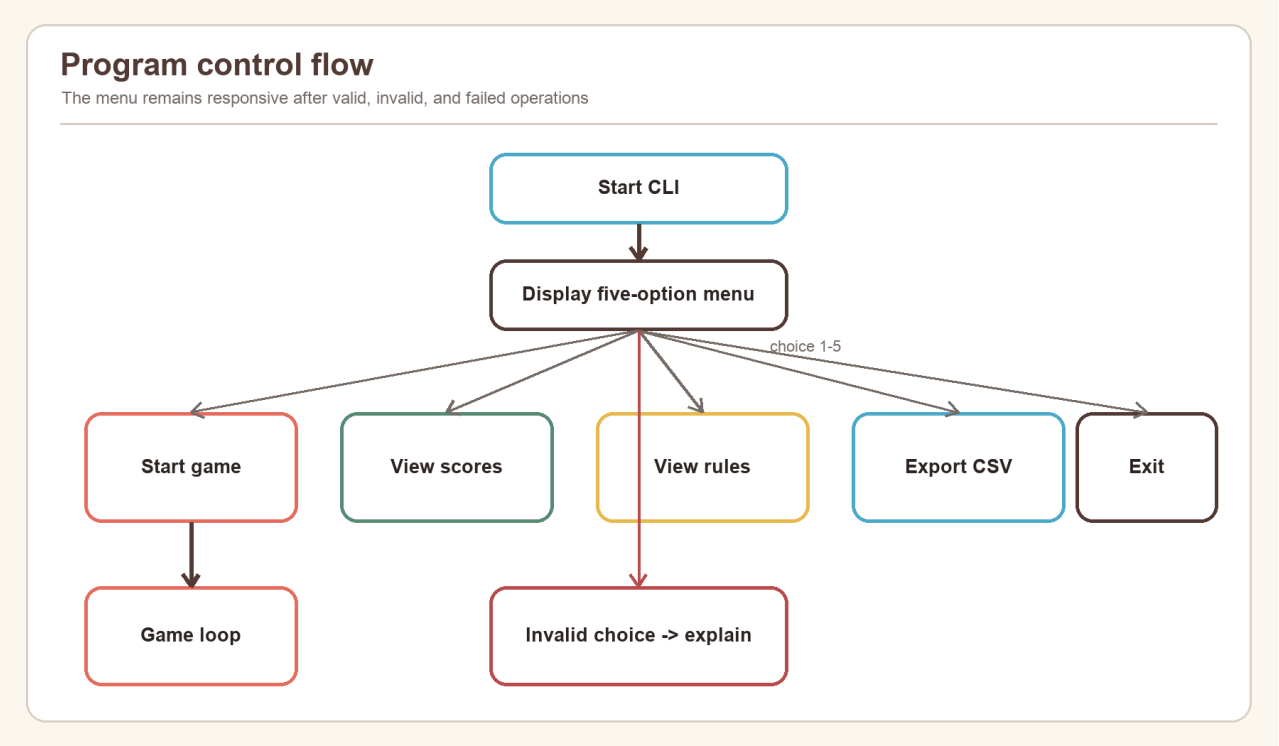

Figure 8. Top-level menu and control-flow design.

4.1: Pre-generation Design

Pre-generation covers everything required before an active code-breaking loop begins: capture the player name, choose a preset or custom configuration, validate all settings, determine the Code Maker, obtain or generate a legal secret, and create an active state. Failure in this stage returns a clear message without creating a partial score record.

4.1.1: Configuration Design

Official settings are stored as GameConfig constants in a dictionary keyed by normalized difficulty name. Every preset is constructed through the same __post_init__ validation used for Custom mode. Custom mode prompts for colour count, code length, maximum attempts, duplicate policy, and human/computer Code Maker. Custom games are marked unranked.

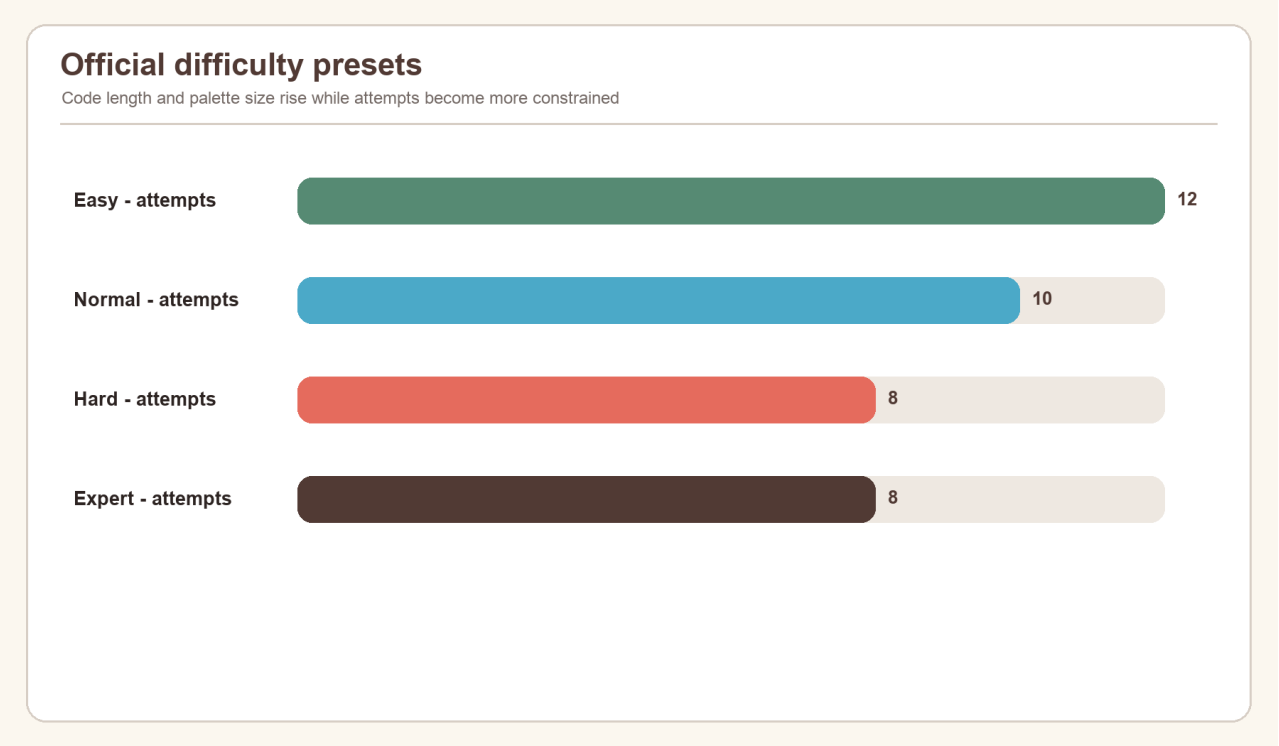

Figure 9. Attempt limits across the four official presets.

Preset	Colours	Code length	Attempts	Duplicates	Ranked
Easy	5	4	12	No	Yes
Normal	6	4	10	Yes	Yes
Hard	8	5	8	Yes	Yes
Expert	10	6	8	Yes	Yes

4.1.2: Input Validation

Validation is designed as a retryable boundary, not a reason for the program to terminate. `_number` loops until an integer falls within the permitted range. `_yes_no` accepts `y`, `yes`, `n`, or `no` after stripping and case folding. A human secret loops until it passes the same code validator used for guesses.

Guess validation is deliberately completed before feedback or attempt construction. This guarantees that malformed separators, wrong length, unknown colours, or prohibited duplicates cannot consume an attempt. Stable error codes make the same rule usable through API boundaries, while the CLI displays concise messages.

Figure 10. Source screenshot: normalization and validation functions.

4.2: Generation Design

Generation is the active game phase. The state begins as CREATED, immediately transitions to ACTIVE, and accepts guesses until it reaches WON, LOST, or ABANDONED. Each accepted guess produces a new immutable GameState rather than modifying the old one.

4.2.1: Teacher Assignment

The required “Teacher Assignment” heading maps to role assignment. The design assigns Code Maker authority from configuration: official presets use the computer; Custom may select a human. The Code Breaker role is the player submitting guesses. This explicit role model avoids ambiguous Boolean flags scattered through the game loop.

CodeMakerType is an enum, and GameMode records whether the game is solo or pass-and-play. Role assignment therefore becomes a typed domain decision that is also persisted in the score record.

4.2.2: Subject Selection

“Subject Selection” maps to selecting the problem difficulty. The CLI shows four official presets and Custom. A dictionary converts menu numbers to preset names, while get_preset normalizes the name and raises UNKNOWN_DIFFICULTY for invalid direct calls.

The design keeps user-facing selection separate from configuration values. This prevents a menu branch from reimplementing palette sizes or attempt counts. One authoritative preset table supports the CLI, tests, and wider product.

4.2.3: Classroom Generator

“Classroom Generator” maps to creating a game session. create_game(config, mode) constructs a GameState and performs the only legal CREATED -> ACTIVE transition. It also records the start timestamp used for persisted timing information, although elapsed time does not award score points.

The function is intentionally small because configuration validation already occurred in GameConfig. This demonstrates design by contract: a valid configuration enters the engine, and an active immutable state leaves it.

4.2.4: Undo Design

The CLI does not provide an undo command for accepted guesses. This is a deliberate integrity rule: feedback changes the player's knowledge, so removing an attempt would allow trial information without cost. The immutable attempt tuple also makes silent deletion unnatural.

Before submission, the player can correct terminal text normally. Invalid submissions are not accepted and therefore do not need undo. Abandonment is explicit and produces zero score. A future practice-only undo could be implemented by creating a new state marked unranked, but it must never alter official results.

4.2.5: Teacher Generator

“Teacher Generator” maps to secret generation. `generate_secret` accepts a `GameConfig` and optional random-source protocol. With duplicates allowed, it chooses independently for each position. Without duplicates, it samples distinct colours. Production defaults to `secrets.SystemRandom`; tests inject a seeded generator for repeatability.

Human-generated secrets pass through `validate_code`. Both paths therefore guarantee correct length, known colours, and compliance with the duplicate rule before play begins.

4.3: Post-generation Design

Post-generation begins when the state becomes terminal. The CLI prints the outcome, calculates or reads the terminal score, constructs a 13-field `ScoreRecord`, and attempts to append it. Persistence failure produces a warning but does not erase the completed game experience.

4.3.1: Printing and Exporting

Printing uses the state as the source of truth. Win text reports attempts used; loss text reveals the secret; abandonment states that no score was awarded. Every case prints the numerical score. The detailed record is then available to the high-score view and export operation.

Atomic export is a stronger design than copying the source file directly. It produces a complete new file before replacing the destination, preventing a crash from leaving a half-written report.

4.3.2: Summary Data

Summary data contains player name, mode, difficulty, Code Maker, colour count, code length, duplicate policy, attempts used, maximum attempts, score, scoring version, result, and UTC timestamp. It excludes the secret and the full guess history because neither is required for local ranking.

The schema supports fair comparisons and later analysis. A future migration can distinguish scoring versions rather than mixing results calculated by different formulas.

4.3.3: Timetable Re-generation

“Timetable Re-generation” maps to starting another game. Returning to the menu and selecting Start creates a fresh configuration, secret, and state. Previous state is not mutated; only its terminal summary remains in CSV.

Fresh generation is important for computer Code Maker games because ordinary play must not reuse a predictable seed. Pass-and-play intentionally allows a new human secret. Regeneration is therefore a new session, not an undo of the former session.

4.4: Overall Design

The overall design can be summarised as “functional core, imperative shell.” The core receives values and returns validated values or new state. The CLI shell performs input/output. The storage adapter performs file effects. Dependency injection supplies input, secret input, output, store, and random source for tests.

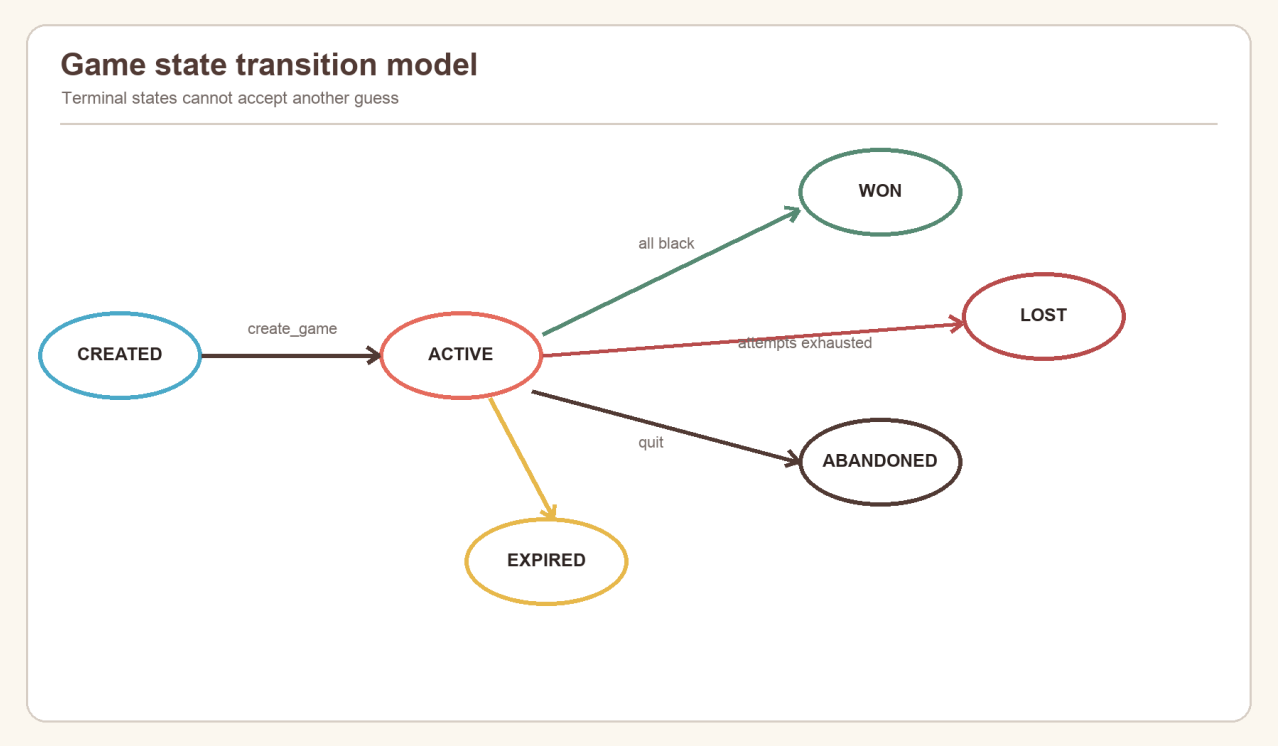

Figure 11. Legal game-state transitions.

This structure produces maintainability and confidence. Pure rule functions are easy to test exhaustively; terminal integration is tested with scripted input; file handling is tested in temporary directories. The same core can be reused by another interface without translating the rules.

5: Design Implementation

Implementation converted the design into three cooperating Python areas: `mastermind_core` for rules and immutable state, `mastermind_cli.app` for the interaction loop, and `mastermind_cli.storage` for local records. The implementation favours explicit values over hidden side effects. A game transition returns a new state; a rejected guess raises a stable domain error; and storage is called only after the result is known.

The source is packaged rather than run as a loose script. The CLI entry point calls `MastermindCLI().run()`, while the project metadata supplies the installed `cipherboard` command. This permits the same code to run in development, automated tests, and packaged desktop installations without changing imports.

5.1: Initialization

Initialization occurs at two levels. Package initialization exports the public domain API from `mastermind_core.__init__`, so consumers do not depend on private module paths. Runtime initialization constructs a `MastermindCLI` with default terminal input, hidden secret input, printed output, and a CSV store at `data/high_scores.csv`. Tests replace those dependencies with in-memory callables and temporary paths.

The first visible action is printing `Mastermind Logic Lab`, followed by the menu. Selecting Start performs a second, game-specific initialization: name, configuration, secret, mode, active state, and attempt history. No result file is created merely by launching the program; it is created only when the first terminal result is appended.

5.1.1: Data Structures

The central data structures are frozen dataclasses with slots. Freezing prevents accidental field assignment, and slots prevent unplanned attributes. Tuples preserve order while making palettes, guesses, and histories immutable. Enums restrict status and role fields to known values.

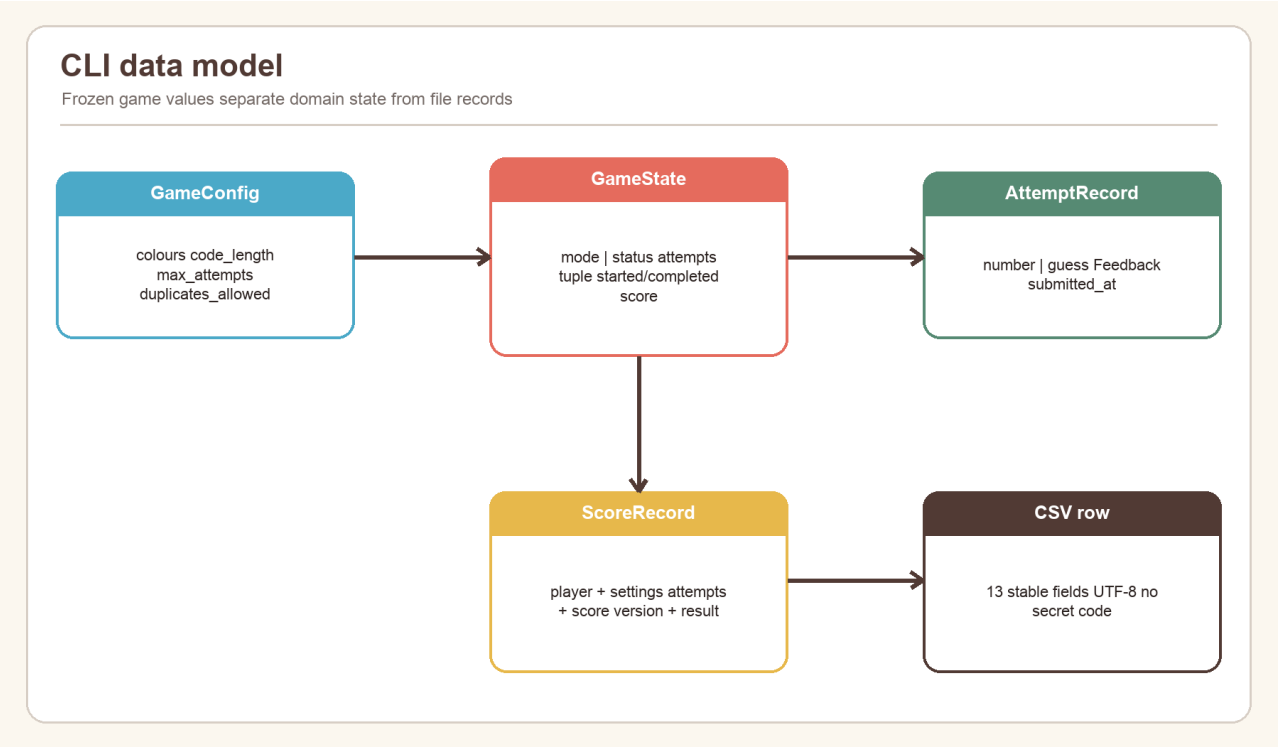

Figure 12. Core data model and relationships used by the CLI.

Structure	Important fields	Responsibility
GameConfig	colours, length, attempts, duplicate rule, Code Maker	Defines and validates the legal game space
Feedback	black, white	Stores one bounded comparison result
AttemptRecord	number, guess, feedback, timestamp	Represents one accepted guess
ScoreBreakdown	attempts, difficulty, time, total, version	Makes scoring auditable

Structure	Important fields	Responsibility
GameState	config, mode, status, attempts, timestamps, score	Holds the complete rule-engine state
ScoreRecord	13 serializable summary fields	Defines the local persistence schema

GameState.attempts_used and attempts_remaining are calculated properties rather than separately stored counters. This eliminates a possible inconsistency between history length and displayed allowance. Likewise, GameStatus.terminal derives terminal membership from the enum rather than asking callers to repeat four comparisons.

5.1.2: Global Variables

Module-level constants are used only for stable policy and schema: COLOUR_IDS, RULE_SET_VERSION, SCORING_VERSION, CSV_FIELDS, the official preset table, and rule text. Mutable game state is not global. Each play_game call keeps its own local state and secret, so a completed or abandoned game cannot leak attempts into the next session.

The constants serve as single sources of truth. COLOUR_IDS controls palette membership, CSV_FIELDS controls both append and export order, and version constants identify the rule and score interpretations. This is safer than scattering literal strings or column lists through menu branches.

5.1.3: Configuration File

Project configuration is declared in pyproject.toml. It defines Python 3.13 compatibility, workspace packages, CLI and GUI executables, linting, type checking, tests, and optional dependencies. .python-version selects the development interpreter, while uv.lock fixes transitive versions for reproducible installation.

Runtime game configuration is represented by GameConfig, not an editable external file. This is intentional: its __post_init__ method normalizes colours and rejects invalid colour counts, duplicate palette choices, unknown identifiers, illegal lengths, attempt limits, insufficient unique colours, and invalid time-bonus bounds. Presets are therefore executable, validated configuration rather than unverified text.

Configuration concern	File or object	Validation point
Python/dependencies	pyproject.toml, uv.lock	uv sync --frozen
Installable commands	project script entries	build/install verification
Test/lint/type policy	pyproject.toml	pytest, Ruff, mypy
Game rules	GameConfig and presets	dataclass construction
Local score location	CSVScoreStore.path	parent creation on append/export

5.2: Classes

MastermindCLI is the application controller. Its constructor accepts four dependencies: normal input, hidden input, output, and score store. Its methods correspond to user tasks rather than low-level rules: run the menu, play a game, build Custom configuration, show scores, and export scores. Helper methods handle repeated numeric and yes/no prompting.

CSVScoreStore is a repository adapter. It has append, read, and export operations and owns every CSV-specific detail. ScoreRecord is its typed transfer object. Neither class knows how feedback is calculated.

Domain dataclasses are intentionally behaviour-light. GameConfig enforces construction invariants; GameState exposes derived attempt counts and legal transitions. Rule functions then operate on these values. This balance avoids a monolithic “Game” class while keeping invariants close to their data.

5.3: Libraries

The runtime relies mainly on the Python standard library, reducing installation size and supply-chain exposure.

Library/module	Use	Reason for selection
collections.Counter	Count unmatched colours	Correct linear-time duplicate handling
secrets.SystemRandom	Generate computer secrets	Operating-system randomness rather than a predictable default PRNG
dataclasses	Typed immutable records	Compact, inspectable domain models
enum.StrEnum	Status, mode, role, visibility	Restricted values that serialize clearly
csv	Read/write local scores	Standards-aware quoting and newline handling
tempfile and os	Safe export and durable flush	Temporary creation, fsync, atomic replacement
getpass	Hide human Code Maker input	Prevent secret echo in normal terminals
pathlib	Cross-platform paths	Avoid manual path-separator logic
pytest	Automated verification	Fixtures, parameterization, temporary directories, clear failures

Ruff and mypy are development tools rather than runtime dependencies. Ruff checks style and likely defects; mypy checks the typed contracts. `uv` resolves and runs the environment consistently across macOS, Windows, local development, and CI.

5.4: Functions

Functions are grouped by lifecycle: pre-generation functions collect or validate setup; generation functions create secrets, evaluate guesses, and transition state; post-generation functions calculate results and persist summaries; the main function coordinates the terminal session. Pure functions are kept independent of printing and file access so their inputs and outputs can be tested directly.

5.4.1: Pre-generation Functions

`normalize_code` accepts a string or sequence. For strings, it trims whitespace, recognizes comma, hyphen, or whitespace separators, and otherwise treats each character as one colour identifier. It uppercases every token and returns a tuple. This means `rbgy`, `R B G Y`, and `r,b,g,y` converge on one representation.

`validate_code` calls normalization and then checks length, palette membership, and the duplicate rule. It raises a `DomainError` containing a stable code and a safe message. `_number` and `_yes_no` implement similar retry boundaries for Custom settings. `get_preset` looks up authoritative validated configurations instead of rebuilding them inside the CLI.

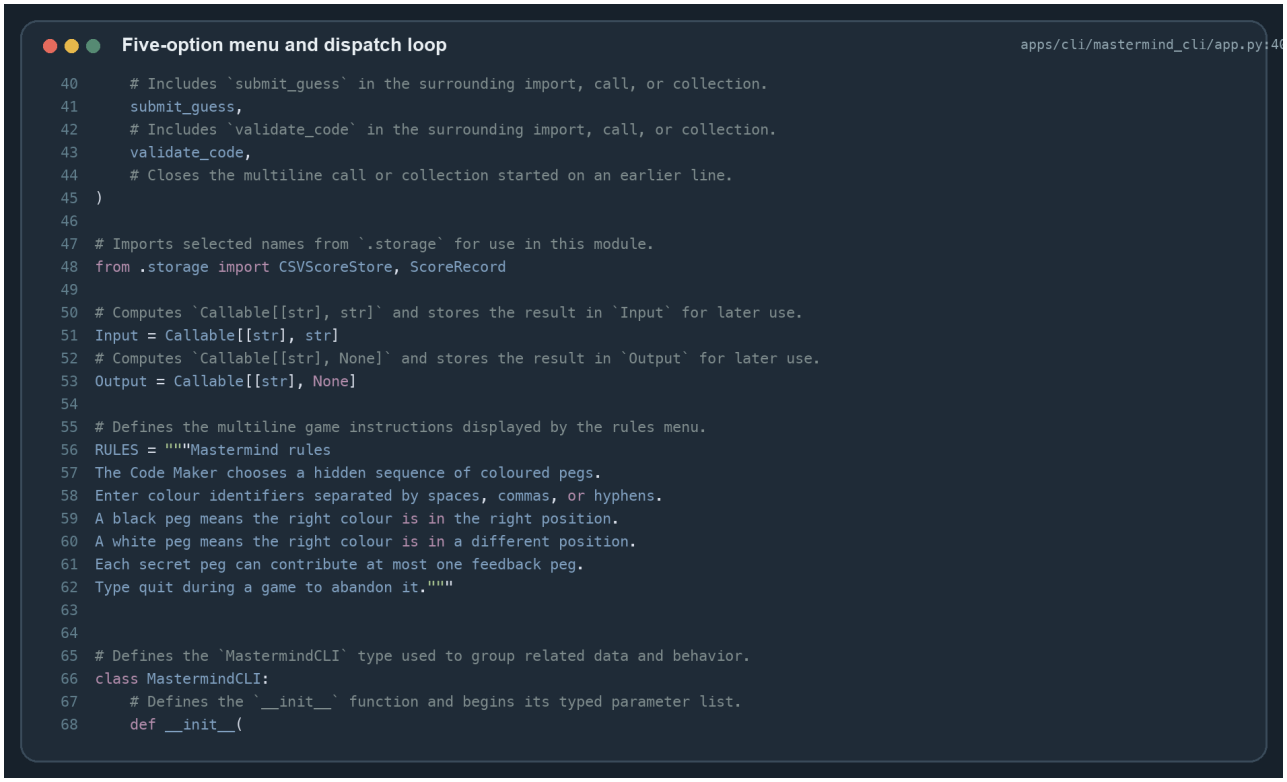

Figure 13. Source screenshot: menu dispatch and difficulty selection.

The pre-generation contract is simple: no GameState is created until all configuration and secret requirements pass. As a result, later functions can operate on a valid configuration and a legal secret.

5.4.2: Generation Functions

generate_secret receives a configuration and an optional random-source protocol. It uses independent choices when duplicates are allowed and sampling when they are forbidden. Tests inject deterministic randomness, while ordinary play uses SystemRandom.

calculate_feedback first marks exact position matches. It then counts only the unmatched secret and guess colours and sums the minimum occurrence count for each colour. Removing black matches before Counter intersection is the key to avoiding double counting.

Duplicate-safe feedback function

packages/mastermind_core/masterm

```python
from collections.abc import Sequence

# Imports selected names from `.errors` for use in this module.
from .errors import DomainError

# Imports selected names from `.models` for use in this module.
from .models import Feedback


# Defines the `calculate_feedback` function and begins its typed parameter list.
def calculate_feedback(secret: Sequence[str], guess: Sequence[str]) -> Feedback:
    # Tests `len(secret) != len(guess)` before running the nested branch.
    if len(secret) != len(guess):
        # Raises the specified exception to report an invalid or failed operation.
        raise DomainError("SEQUENCE_LENGTH_MISMATCH", "Secret and guess lengths must match.")
    # Computes `0` and stores the result in `black` for later use.
    black = 0
    # Computes `[]` and stores the result in `unmatched_secret` for later use.
```

Figure 14. Source screenshot: duplicate-safe black and white feedback.

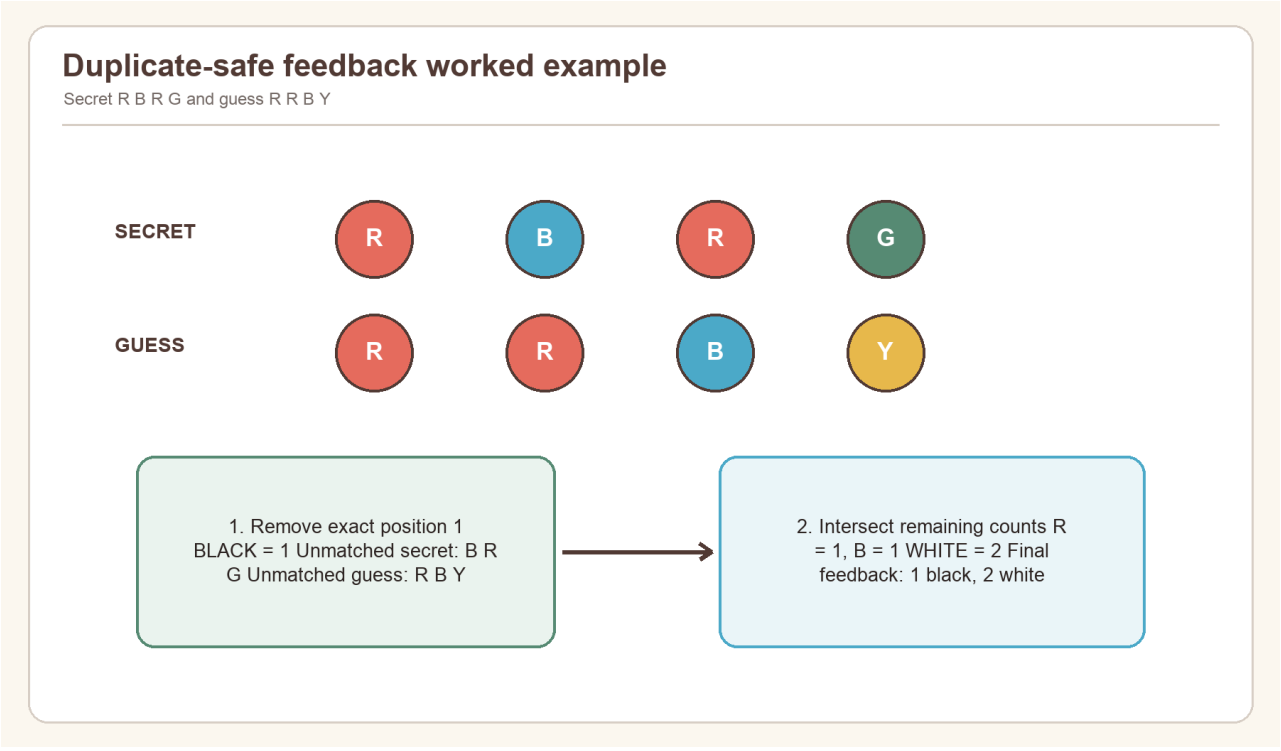

Figure 15. Worked feedback example showing exact-match removal before colour counting.

If the secret is R R B G and the guess is R B R Y, position 1 is black. The unmatched secret contains R, B, G; the unmatched guess contains B, R, Y. Their count intersection contains one R and one B, producing two whites. The result is therefore 1 black and 2 white, never 1 black and 3 white.

submit_guess is the transactional rule boundary. It rejects non-active state, validates the guess, calculates feedback, appends a numbered and timestamped record, determines whether the result is won or lost, and attaches a score only for terminal state. Because the original dataclass is frozen, the function uses replace to return a new state.

Immutable attempt transition packages/mastermind_core/masterm

```python
from .models import AttemptRecord, GameConfig, GameMode, GameState, GameStatus

# Imports selected names from `.scoring` for use in this module.
from .scoring import calculate_score

# Imports selected names from `.validation` for use in this module.
from .validation import validate_code


# Defines `RandomSource` as a specialization of `Protocol`.
class RandomSource(Protocol):
    # Defines the `choice` function and begins its typed parameter list.
    def choice(self, sequence: Sequence[str]) -> str: ...

    # Defines the `sample` function and begins its typed parameter list.
    def sample(self, population: Sequence[str], k: int) -> list[str]: ...


# Defines the `generate_secret` function and begins its typed parameter list.
def generate_secret(config: GameConfig, rng: RandomSource | None = None) -> tuple[str, ...]:
    # Computes `rng or secrets.SystemRandom()` and stores the result in `source` for later use.
    source: RandomSource = rng or secrets.SystemRandom()
    # Tests `config.duplicates_allowed` before running the nested branch.
    if config.duplicates_allowed:
        # Returns `tuple(source.choice(config.colours) for _ in range(config.code_length))` to
        # the caller as this function's result.
        return tuple(source.choice(config.colours) for _ in range(config.code_length))
    # Returns `tuple(source.sample(config.colours, config.code_length))` to the caller as this
    # function's result.
```

Figure 16. Source screenshot: accepted-guess transaction and terminal-state selection.

Scoring is versioned as score_v1. A non-win is zero. A win receives an attempts component of (maximum attempts - attempts used + 1) × 100, plus a difficulty component of code length × 50 + colour count × 20 + 100 when duplicates are allowed. Time is stored for compatibility but awards no points, which avoids favouring faster typists over careful reasoning.

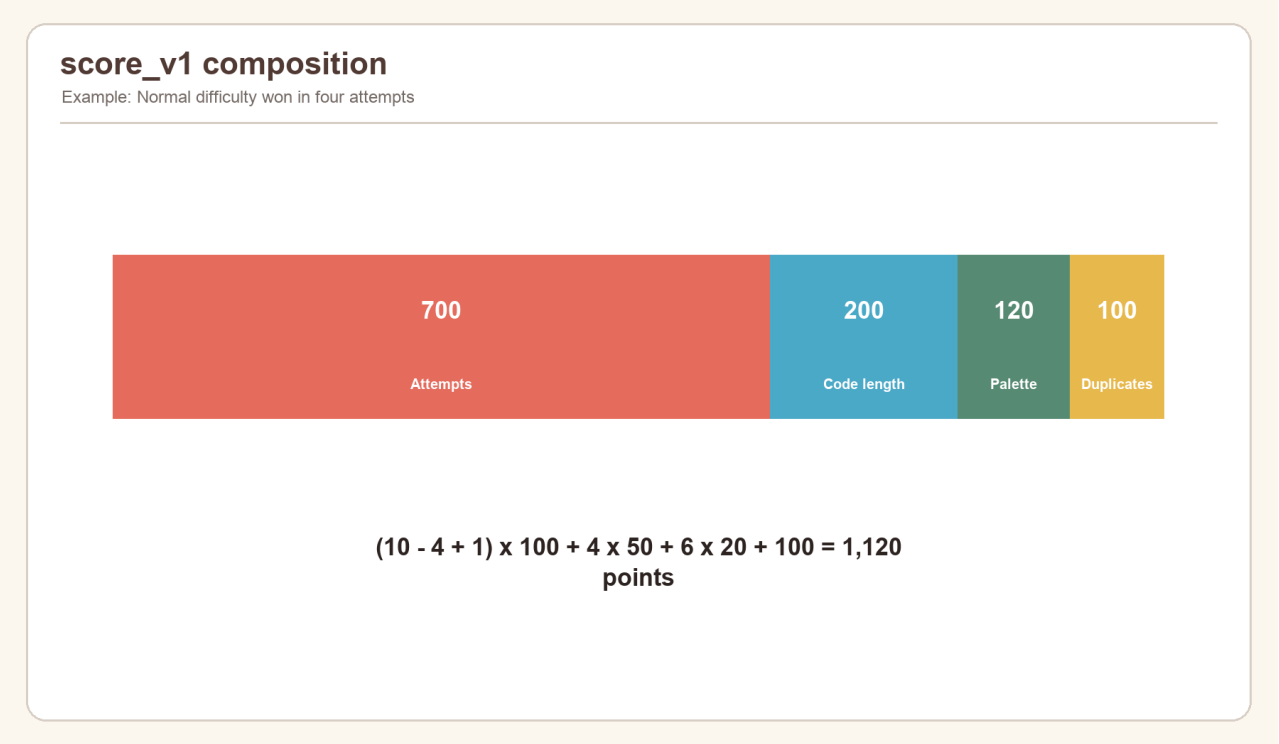

Figure 17. Score composition for representative preset wins.

5.4.3: Post-generation Functions

At terminal state, the CLI prints a status-specific message and score. It constructs ScoreRecord.now(...) with UTC timestamp, mode, difficulty, Code Maker, configuration summary, attempts, score version, and result. CSVScoreStore.append creates the parent directory, writes a header when needed, sanitizes spreadsheet-formula prefixes, flushes, and calls fsync.

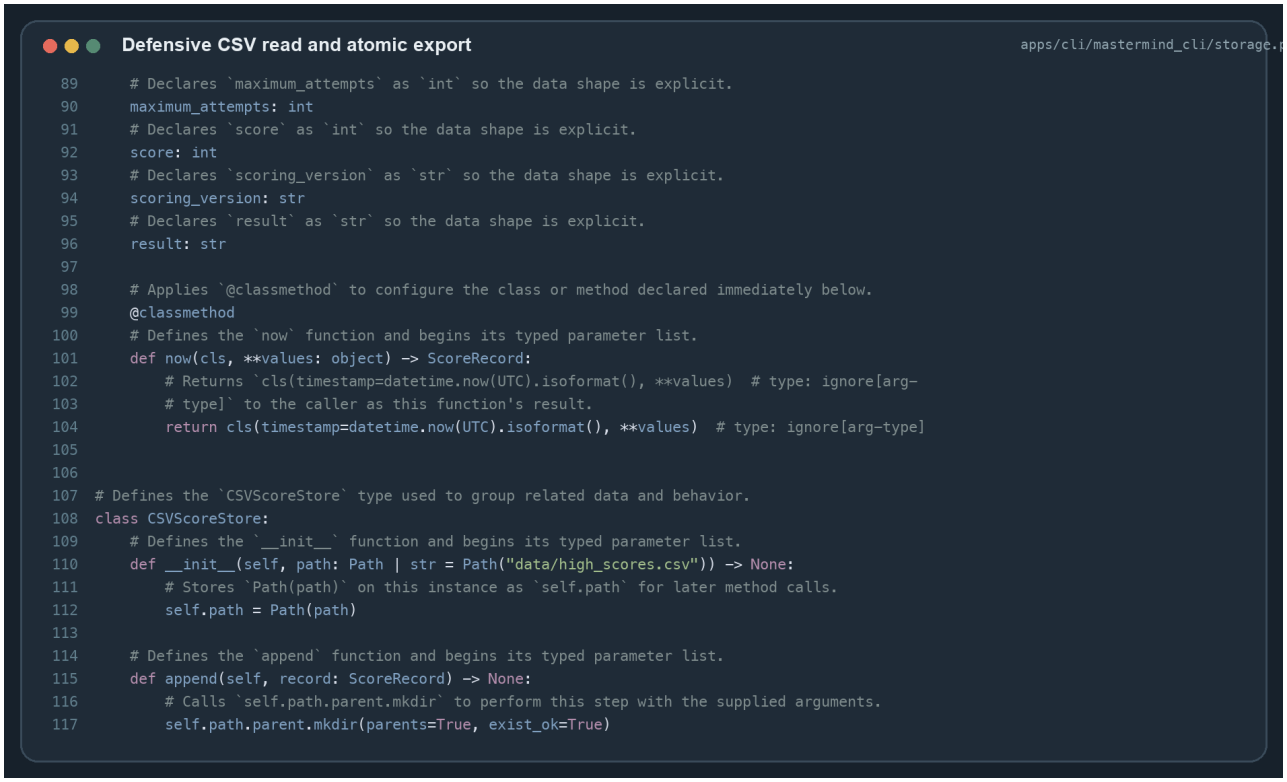

Figure 18. Source screenshot: CSV sanitization, defensive reading, and atomic export.

`read` verifies the header before parsing. Each malformed row is skipped with a runtime warning rather than preventing all valid rows from loading. Records are sorted deterministically. `export` writes the canonical header and validated records to a temporary sibling file, synchronizes it, and calls `os.replace`. If any exception occurs, it removes the temporary file and preserves the former destination.

5.4.4: Main function

The installed entry point delegates to `MastermindCLI().run()`. The main loop prints the menu, trims the user's choice, dispatches choices 1–5, and repeats until Exit returns status code 0. A game returns to the same menu after success, loss, abandonment, or a recoverable save warning.

The executable wrapper catches end-of-input and keyboard interruption at the process boundary. This is the correct layer because these signals may arrive at any prompt. Normal menu and rule methods remain focused on application behaviour rather than duplicating process-level exception handling.

The following pseudocode summarizes the implementation:

```
main
  construct CLI with terminal and CSV dependencies
  repeat
    print menu and read choice
    if Start: configure, generate, play, score, save
    if Scores: read, sort, print first 20
    if Rules: print rules
    if Export: validate records, write atomic CSV
    if Exit: return success
    otherwise: explain valid range
```

6: Testing & Evaluation

Testing was performed in three phases: environment and boundary readiness, rule and integration correctness, and end-user output acceptance. Automated evidence was refreshed from repository baseline 67b4cc9be06 961 483 661bf3 219 626a4ea2e6014e on 1 September 2026. The successful GitHub Actions run executed 117 Python tests in 19.92 seconds with 99.72% branch-aware coverage of the canonical core, plus 36 browser end-to-end tests with 8 intentional browser/project skips. Ruff and mypy also passed across the complete Python source set. MongoDB and Redis are started by CI for service-level tests; the CLI-focused tests remain independent of those services.

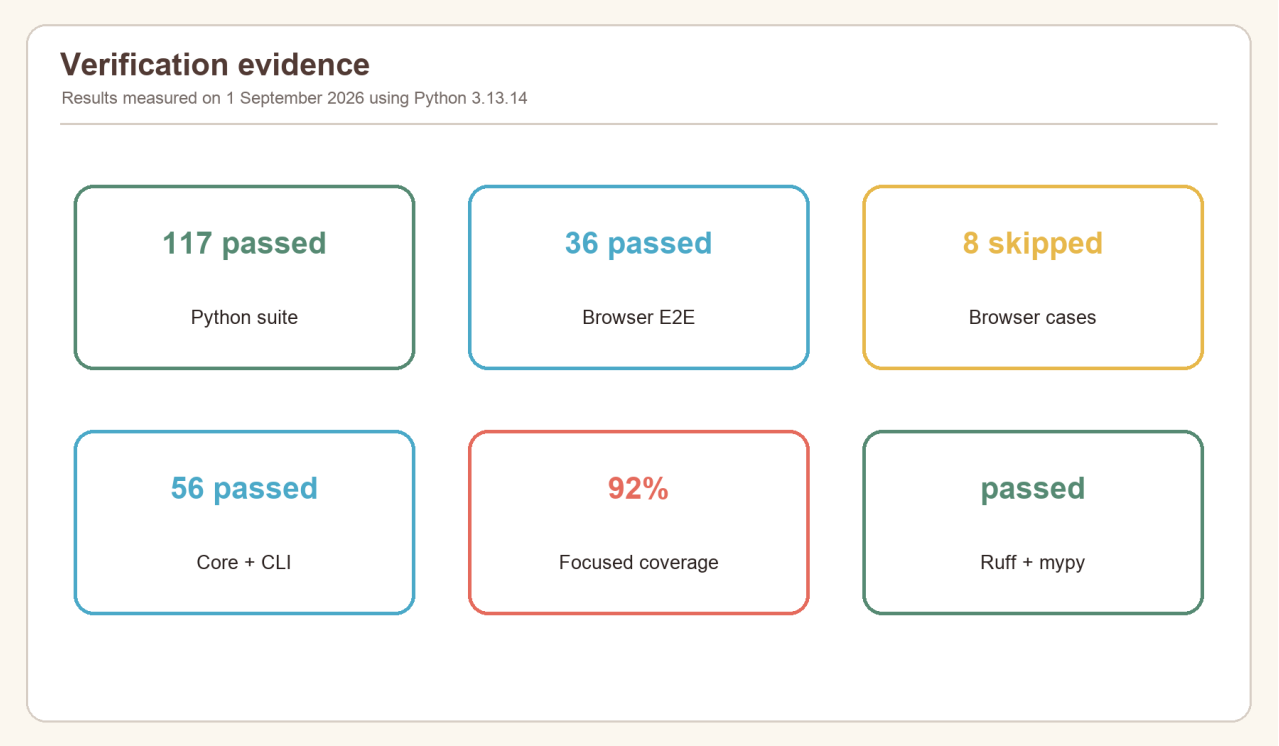

Figure 19. Measured verification result across the repository test suite.

6.1: Testing plan

The plan combines black-box and white-box techniques. Black-box cases exercise menu choices, valid and invalid user inputs, visible feedback, stored records, and exports. White-box cases target every branch in validation, feedback, state transitions, scoring, and persistence. Temporary directories isolate file tests; injected input and output isolate terminal tests; deterministic random sources isolate generation tests.

Test level	Main purpose	Representative oracle
Import/configuration	Confirm environment and invariant construction	Import succeeds or stable error code appears
Unit	Verify one rule independently	Exact feedback tuple, state, or score breakdown
Integration	Verify components cooperate	Scripted CLI output and temporary CSV
Constraint	Prove invalid data cannot corrupt state	Same attempt count after rejection
System/output	Verify actual commands and artifacts	Exit code, table content, exported header
Acceptance	Judge task completion from player perspective	Complete game without technical knowledge

Pass criteria were defined before interpretation: all focused core and CLI tests pass; Ruff reports no findings; mypy reports no errors; accepted inputs produce the expected state; rejected inputs preserve state; persistence either succeeds completely or reports a recoverable warning.

6.2: Testing Phase 1

Phase 1 verifies that the program can start in a controlled environment and that its outer boundaries reject invalid setup. This prevents later gameplay results from being confused with installation or configuration faults.

6.2.1: Library Import Test

The project environment was synchronized from the lockfile and the CLI/core modules were imported through pytest discovery. The complete suite collected and executed successfully. Ruff also resolved the same package structure, and mypy checked 13 focused source files without import errors.

Case	Action	Expected	Observed
P1-01	Import mastermind_core	Public models and functions available	Pass
P1-02	Import mastermind_cli	CLI and storage exports available	Pass
P1-03	Invoke module entry point	Menu begins without traceback	Pass through CLI tests
P1-04	Run typed analysis	Package imports and annotations resolve	13 files passed

6.2.2: Configuration File Test

Configuration tests construct every official preset and invalid boundary combinations. The expected result is either a normalized immutable object or a specific DomainError.

Case	Input	Expected result	Result
P1-05	Easy, Normal, Hard, Expert	Valid documented values	Pass
P1-06	4 colours	INVALID_COLOUR_COUNT	Pass
P1-07	repeated palette identifier	DUPLICATE_COLOUR_OPTION	Pass
P1-08	code length 7	INVALID_CODE_LENGTH	Pass
P1-09	21 attempts	INVALID_MAX_ATTEMPTS	Pass
P1-10	3 colours for length 4 without repeats	INSUFFICIENT_UNIQUE_COLOURS	Pass

6.2.3: Command Input Test

Scripted callables supplied menu choices and guesses without requiring an interactive terminal. Tests confirm whitespace trimming, invalid-choice recovery, case-insensitive quit, default player name, and accepted guess notations. Output was collected as strings and compared with player-visible messages.

Input	Classification	State effect
RBGY	valid compact code	one accepted attempt
r b g y	valid separated lowercase	normalized and accepted
R,B,G,Y	valid comma-separated	normalized and accepted
R--B	empty separated token	rejected, no attempt
RBG for length 4	wrong length	rejected, no attempt

Input	Classification	State effect
quit	abandonment command	terminal abandoned state

6.2.4: Parameters Modification Test

Custom-mode tests modify colour count, length, attempts, repetition policy, and Code Maker. Boundary values 5/10 colours, 3/6 positions, and 1/20 attempts are accepted. Values immediately outside the range cause a retry. Yes/no input accepts full words and single letters but rejects ambiguous responses.

The most important combined case selects no duplicates with a human Code Maker. The hidden secret must be valid under the newly created configuration. This proves that parameter changes are used by subsequent validation rather than merely printed.

6.3: Testing Phase 2

Phase 2 verifies the computational core and integration between core, CLI, and local storage. The refreshed focused command covering core and CLI tests reported 56 passed in 0.53 seconds and 92% measured coverage for the selected source set. The canonical core reached 99% branch-aware coverage because one defensive branch remained partial; CLI storage reached 97%, the process entry point 83%, and the interactive application 75%.

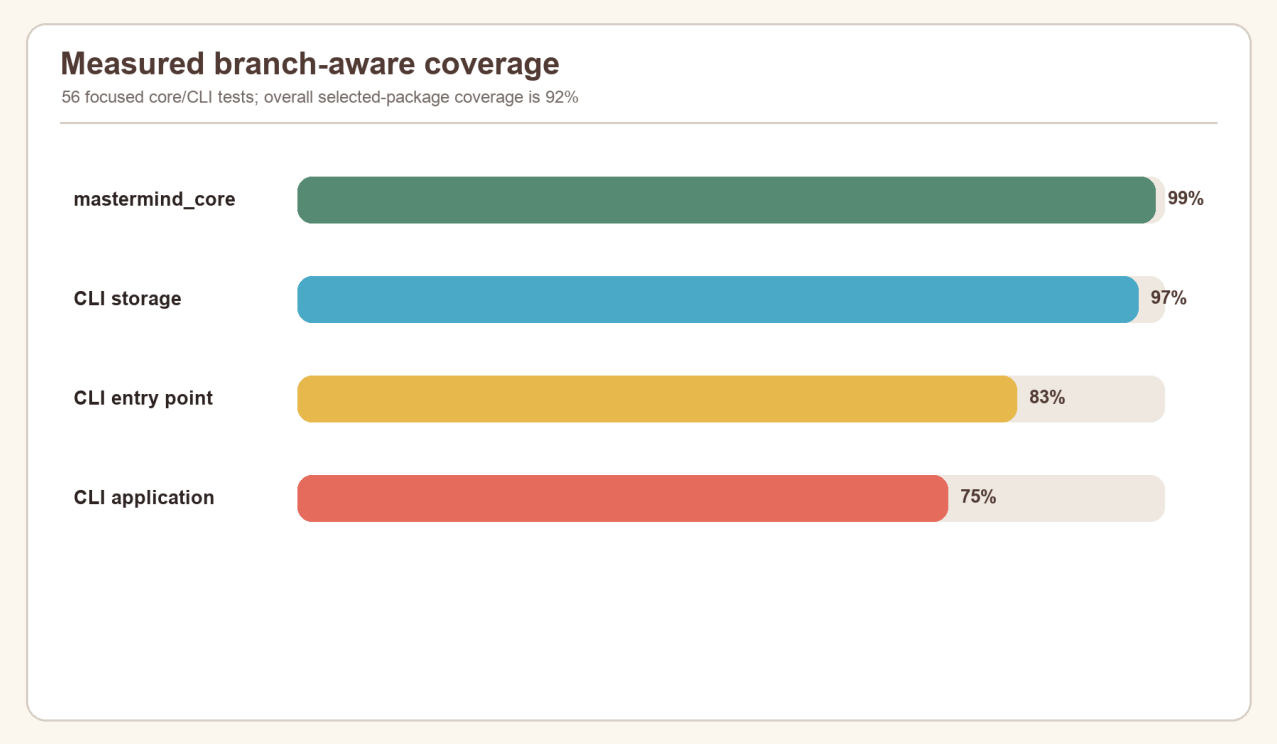

Figure 20. Measured statement coverage for the focused CLI and core verification.

6.3.1: Unit Tests

Feedback unit tests cover all black, all white, mixed, no match, and duplicate-heavy cases. Validation unit tests cover each separator and error code. Engine tests cover secure generation rules, active-state requirements, immutable history growth, win, final-attempt win, loss, abandonment, and illegal terminal submissions. Scoring tests separately verify win components and zero-score statuses.

Examples of strong unit oracles include:

- secret R B G Y, guess R B G Y gives 4 black and 0 white;
- secret R B G Y, guess Y G B R gives 0 black and 4 white;
- secret R R B G, guess R B R Y gives 1 black and 2 white;
- a winning final attempt is WON, not LOST;
- a non-winning final attempt is LOST with zero score; and

- submission to a won, lost, or abandoned state raises an illegal-transition error.

6.3.2: Integration Tests

Integration tests instantiate `MastermindCLI` with scripted input, captured output, and a `CSVScoreStore` in `tmp_path`. They exercise the real controller rather than mocking the core. One scenario plays to a win and verifies the printed attempt table and saved result. Others test invalid setup, pass-and-play, score display, export, and storage warnings.

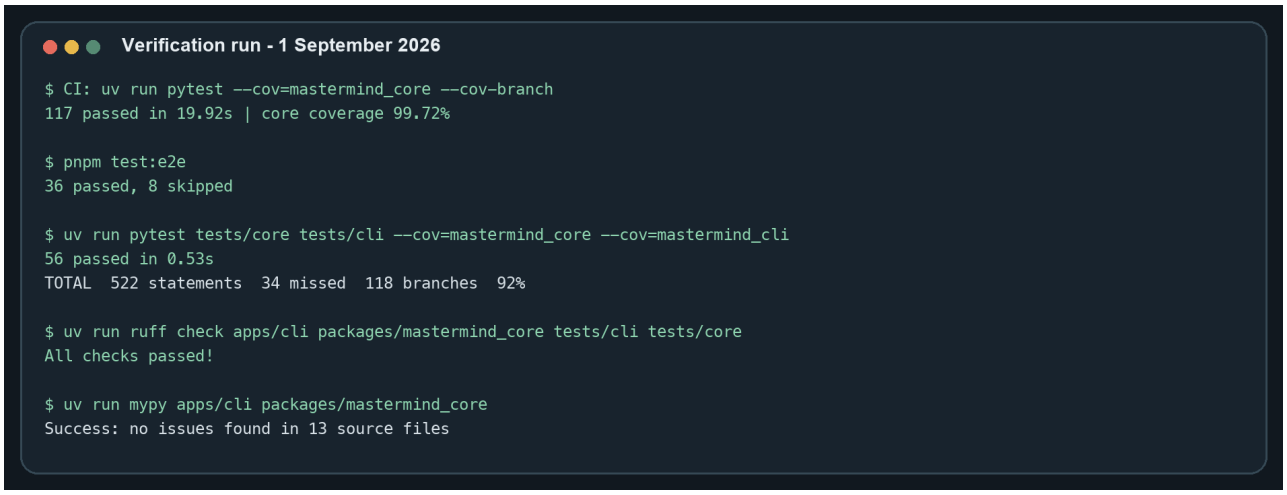

Figure 21. Terminal verification commands and observed pass counts.

This layer is essential because unit tests cannot detect a disconnected menu option, incorrect prompt loop, or omitted persistence call. Conversely, it remains deterministic because the secret input and storage location are injected.

6.3.3: Constraint Checking

Constraint checks test both boundaries and invariants after operations. The most important invariant is that an invalid guess leaves `attempts_used`, `attempts_remaining`, status, and history unchanged. Tests also confirm that feedback totals never exceed code length, white matches cannot reuse black matches, and generated secrets respect palette and duplicate rules.

For persistence, malformed headers yield an empty result with a warning, malformed rows are skipped individually, and export always contains the canonical header. Player names beginning with `=`, `+`, `-`, `@`, tab, or carriage return are escaped before spreadsheet use. These are integrity constraints, not cosmetic behaviours.

6.4: Testing Phase 3

Phase 3 evaluates complete outputs and user tasks. Its focus is not internal branches but whether a player can understand the program, complete a game, find the result, and reuse the recorded information.

6.4.1: Prints & Exports Tests

Print tests assert the five menu choices, rules text, configuration summary, attempt-table labels, pluralization, terminal status message, score, and export confirmation. Export tests read the actual target with `csv.DictReader`, confirming a 13-column header and values that round-trip correctly.

The default export works with an empty store and still creates a usable header-only CSV. An explicit nested destination creates parent directories. A simulated failure leaves the previous destination intact and removes the temporary file. These behaviours make the output suitable for spreadsheet analysis without risking silent data loss.

6.4.2: Summary Report Test

The high-score summary is tested with multiple records. Expected ordering is score descending, attempts ascending, timestamp ascending. The display is limited to 20 records, names are truncated to protect table alignment, and an empty store prints `No local scores yet.`

The detailed CSV summary retains configuration context, so the same numerical score is not detached from difficulty or scoring version. It intentionally excludes the secret, which reduces privacy and fair-play risk.

6.4.3: Re-Generation Test

Re-generation is tested by starting new game sessions from the menu. Every call to `create_game` yields empty history, active status, and a new start time. Computer-generated secrets are requested anew; human mode requests a new hidden secret. Completed state remains only in the saved summary.

Deterministic generators are used only inside tests. Production generation does not use a fixed seed. This distinction allows exact testing without weakening ordinary play.

6.4.4: User Acceptance Test

Acceptance criteria were evaluated as an end-to-end checklist.

User story	Acceptance criterion	Status
New player	Can read rules and start Easy without configuration knowledge	Accepted
Experienced player	Can select Expert or configure Custom constraints	Accepted
Two local players	Can enter a hidden secret and hand over the device	Accepted
Typing mistake	Receives corrective message without losing an attempt	Accepted
Completed player	Sees result, score, and saved local record	Accepted
Returning player	Can view ranked local summaries	Accepted
Data user	Can export standards-compliant CSV	Accepted
Interrupted user	Exits without an unhandled traceback	Accepted

The main usability limitation is the text-only representation of colours. Letter identifiers are unambiguous and accessible in monochrome, but new players must learn the legend. The GUI offers a more visual alternative while sharing the same rules.

7: Debugging & Improvements

This section records seven concrete defect classes and the implementation safeguards that resolve them. Because the current repository is the verified baseline, the entries are a reconstruction from code boundaries and regression tests rather than a claim that each defect corresponds to a separately preserved historical commit.

7.1: Error List

Error	Symptom/risk	Root cause	Resolution
1	Too many white pegs with duplicates	All colours counted before exact matches removed	Remove black positions, then intersect Counter values
2	Invalid guess consumes an attempt	State updated before validation completes	Validate first; construct record only on success
3	Human secret visible to Code Breaker	Ordinary echoed input used	Use getpass and a 40-line hand-off clear
4	CSV opens as an active formula	Untrusted cells begin with formula marker	Prefix dangerous leading characters with apostrophe
5	One damaged row breaks score screen	Entire file parsed as all-or-nothing	Validate header; warn and skip malformed rows
6	Failed export leaves partial target	Destination written directly	Write, flush, sync, then atomically replace
7	Simultaneous duel guesses conflict or overwrite state	Two API workers mutate one room document concurrently	Serialize room attempts with a Redis lock and regression-test concurrent submissions

7.1.1: Error 1, 2 Debugging

Error 1 appeared in reasoning about repeated colours. A naïve algorithm could count one peg as both black and white. The diagnostic case `secret R R B G` against `guess R B R Y` makes the defect obvious. Matching position 1 must be removed, leaving two colour-only intersections. The final algorithm separates exact-match indexes and compares only unmatched counters.

Error 2 concerned operation order. If a record were appended before `validate_code`, wrong-length or disallowed-duplicate guesses would unfairly reduce the attempt allowance. The corrected transaction validates and calculates feedback before it creates the next immutable history. Regression tests compare the entire pre- and post-error state.

7.1.2: Error 3, 4 Debugging

Error 3 affected pass-and-play confidentiality. A normal `input` prompt echoes the secret and remains visible in terminal history. The fix injects `getpass` for hidden entry, validates through the normal rule path, emits blank lines, and prints a hand-off instruction. This is adequate for one shared terminal, though separate-device play would be stronger.

Error 4 affected exported data. Spreadsheet programs may interpret cells beginning with `=`, `+`, `-`, `@`, tab, or carriage return as formulas. Player names are user-controlled, so ordinary CSV quoting alone is insufficient. `csv_safe` prefixes these cells with an apostrophe during append and export. A dedicated test asserts the escaped representation.

7.1.3: Error 5, 6 Debugging

Error 5 concerned damaged persistence. A missing header or one non-integer field should not crash menu option 2. The final reader checks required columns once, converts each row in its own guarded block, emits warnings, and returns the remaining valid records. File, encoding, and CSV errors return an empty list safely.

Error 6 concerned interruption during export. Directly opening the requested path in write mode can erase an existing report before new writing succeeds. The corrected sequence uses `mkstemp` in the target directory, writes and synchronizes the complete dataset, and calls `os.replace`, whose same-filesystem replacement is atomic. An exception handler removes the temporary file.

7.1.4: Error 7 Debugging

Error 7 was found in the wider GUI/API path that shares the Mastermind engine. Two players could submit duel guesses at almost the same moment, allowing independent MongoDB transactions to contend for the same room and game state. The repair at commit `13fa42f0d9703f945881610a9a2d89ac7d31446f` wraps the authoritative room-to-game mutation in a Redis lock named for the room. The five-second lease and blocking timeout bound failure, while the inner transaction preserves idempotency, attempt order, tie-window logic, and completion events.

Dedicated regression cases submit concurrent guesses, verify that both accepted attempts are retained exactly once, and confirm that retries remain idempotent. This repair does not change the standalone CLI's local game loop, but it matters to the installed GUI and therefore to the combined v1 package described in the user manual.

7.2: Improvement List

The implemented program is reliable, but evaluation identified improvements in maintainability, portability, and user experience.

Priority	Improvement	Expected benefit	Main trade-off
1	Keep domain models typed and immutable	Prevent state drift; easier testing and GUI reuse	More explicit transition code
2	Harden local persistence and add migration metadata	Safer long-term score history	Additional schema/version handling
3	Expand dependency injection and interaction coverage	Reproduce prompts and failures precisely	More test fixtures
4	Add accessible coloured terminal symbols optionally	Faster visual scanning	Terminal capability detection
5	Offer JSON export beside CSV	Richer nested history analysis	Another documented format
6	Separate-device pass-and-play	Stronger secret confidentiality	Requires networking/authentication

7.2.1: Improvement 1, 2, 3

Improvement 1 is already substantially realized through frozen dataclasses, tuples, enums, derived counters, and replacement-based transitions. Further work could introduce a small command/result type so every accepted or rejected action is represented without relying on exceptions at API edges.

Improvement 2 is partly realized through schema headers, scoring version, formula protection, warnings, `fsync`, and atomic replacement. A future release could add an explicit CSV schema version and automatic backup before migration. It could also lock the score file when two CLI processes append simultaneously.

Improvement 3 is realized through injectable normal input, secret input, output, store, and random source. Coverage shows the interactive controller still has more unvisited paths than the pure core. Additional parameterized scripts for every Custom prompt, every storage warning, EOF, and interruption would reduce this gap.

7.3: Additional Feature

The strongest additional feature is architectural reuse. The CLI is not an isolated school script: it uses the same canonical engine as the API and GUI-facing system. Improvements to feedback, validation, scoring, and configuration therefore remain consistent across interfaces.

Other additions include pass-and-play with hidden secret input, four official presets plus Custom, a local high-score table, versioned score records, CSV formula protection, atomic export, malformed-data recovery, and cross-platform installers that include both CLI and GUI entry points. The maintained v1.0.0 archives are rebuilt from the repaired source and contain RELEASE.txt with the exact source commit, allowing the installed payload to be audited. These features extend the minimum expectation without obscuring the game loop.

7.4: Documentation

Documentation exists at three levels. Terminal rules explain play at the moment of need. The repository README explains installation, development commands, architecture, testing, and packaging. This SBA report documents the analysis, algorithms, evidence, evaluation, and user procedure in a form suitable for assessment.

7.4.1: User Manual

Installed application

1. Run the signed or locally built installer for macOS or Windows.
2. Choose the installation destination when prompted. The installer reports the GUI, CLI, service helper, bundled runtime, and supporting files it installs.
3. Open RELEASE.txt in the extracted archive when build provenance is required; the refreshed package identifies v1.0.0 and its full source commit.
4. Start the GUI from the Applications folder or Start menu for a graphical game.
5. Open Terminal, PowerShell, or Command Prompt and enter cipherboard for the CLI.
6. If background services are required by the installed build, use cipherboard-services start, confirm with cipherboard-services status, inspect with cipherboard-services logs, and stop with cipherboard-services stop.

Development environment

From the repository root, install the locked environment and launch the CLI:

```
uv sync --frozen --all-extras
uv run python -m mastermind_cli
```

To verify the implementation:

```
uv run pytest -q
uv run ruff check packages/mastermind_core apps/cli tests/core tests/cli
uv run mypy packages/mastermind_core apps/cli
```

Playing a game

1. Enter 1 at the main menu.
2. Enter a player name, or press Enter to use Player.
3. Select Easy, Normal, Hard, Expert, or Custom.
4. In Custom mode, answer every bounded prompt. For human Code Maker, type the hidden secret and hand over the device.
5. Enter guesses using enabled letters. RBGY, R B G Y, R,B,G,Y, and R-B-G-Y are equivalent when those colours are enabled.
6. Read black as correct colour and position; read white as correct colour but different position.
7. Continue until the code is broken or attempts reach zero. Enter quit to abandon with zero score.

Scores and exports

Menu option 2 prints up to 20 local results. Menu option 4 exports all readable records to mastermind_scores.csv by default or to a path you provide. The working score file is data/high_scores.csv relative to the launch location unless the packaged application configures another data directory. Do not edit a live score file while a game is being saved.

Troubleshooting

- If cipherboard is not found, reopen the terminal after installation and confirm the selected installation directory is on PATH.
- If hidden input seems blank, this is expected: type the secret and press Enter.

- If a guess is rejected, confirm its length, enabled letters, and duplicate policy; the attempt has not been consumed.
- If a save/export warning appears, check directory permissions and free space. The game result is still displayed.
- If a damaged CSV is detected, preserve a copy before editing; valid rows may still load while malformed rows are skipped.

7.4.2: README File

The repository README.md is the developer and operator landing page. It describes Cipherboard, the shared-engine architecture, quick-start commands, applications, quality checks, data services, and packaging. The installer documentation adds platform-specific build and signing instructions. This report complements rather than duplicates the README: it explains why the CLI was designed this way and presents assessment evidence.

8: Conclusion

The Mastermind Game System meets its CLI-centred objective. It supports complete play, readable feedback, configurable constraints, secure computer or human secrets, immutable state, deterministic scoring, recoverable local persistence, and user-controlled export. The result is small enough to understand but structured enough to serve more than one interface.

8.1: Self-Reflection

The most important lesson was that apparently simple rules contain difficult boundary cases. Duplicate feedback required deliberate decomposition; input validation required preserving state on rejection; persistence required considering malicious spreadsheet content and interrupted writes. Designing those boundaries first produced code that was easier to test and explain.

I also learned to separate evidence from appearance. A polished terminal table is useful, but correctness comes from invariants and repeatable tests. Likewise, a high coverage value is supportive rather than conclusive: test cases must still assert the right behaviour. Writing this report exposed where the code is strong and where interaction coverage can grow.

8.2: Program Evaluation

The program's principal strengths are correctness, modularity, and failure handling. Duplicate-safe feedback is linear in code length; game objects are immutable; scoring is auditable and versioned; storage uses canonical fields and atomic export; and the CLI has no network requirement. The shared core also prevents rule divergence between CLI and GUI.

Limitations remain. Local CSV is not designed for concurrent multi-process writing. Pass-and-play privacy depends on one terminal hand-off. The CLI uses letters rather than optional ANSI colour, and only summary data—not full attempt history—is exported. The interactive controller's 75% focused coverage is lower than the core's 99% branch-aware coverage. None prevents the stated use case, but each defines a credible next iteration.

8.3: Future Improvements

Future work should prioritize an explicit storage schema version with migration and locking, broader controller-path tests, optional accessible ANSI styling, JSON attempt-history export, and separate-device Code Maker/Code Breaker sessions. Packaging should add automated signing and notarization verification on macOS and signed installer validation on Windows.

The scoring model could also be evaluated with real player data. Because `score_v1` excludes speed, it rewards reasoning and difficulty but may need recalibration across presets. Any change should create `score_v2`, preserve old records, and avoid retroactively comparing incompatible totals.

8.4: Summary

This project demonstrates the complete development cycle: a precise problem, parameter and constraint analysis, modular design, typed implementation, layered testing, debugging, user documentation, and critical evaluation. The refreshed baseline—117 passing Python tests, 36 passing browser tests with 8 intentional browser skips, focused 92% CLI/core coverage, clean Ruff checks, and clean mypy checks—supports the conclusion that the CLI is fit for its documented local use and that the combined v1 GUI/CLI package includes the repaired service path.

9: Appendix

The appendix records the reference basis and development schedule. Source-code figures in this report are generated from the repository baseline, and measured charts come from the verification commands stated in Section 6.

9.1: References

References were selected for software-development fundamentals, Python standard-library behaviour, test methodology, environment management, and the official assessment context. Websites were accessed on 1 September 2026.

9.1.1: Books

1. Brookshear, J. G., and Brylow, D. *Computer Science: An Overview*. 13th ed., Pearson, 2019.
2. Downey, A. B. *Think Python: How to Think Like a Computer Scientist*. 2nd ed., O'Reilly Media, 2015.
3. Sommerville, I. *Software Engineering*. 10th ed., Pearson, 2015.

9.1.2: Websites

1. Hong Kong Examinations and Assessment Authority. “Hong Kong Diploma of Secondary Education Information and Communication Technology: School-based Assessment Teachers' Handbook (2028 Examination).” https://www.hkeaa.edu.hk/DocLibrary/SBA/HKDSE/SBAhandbook-2028-ICT-E.pdf
2. Hong Kong Examinations and Assessment Authority. “2025 HKDSE Information and Communication Technology Assessment Framework.” https://www.hkeaa.edu.hk/DocLibrary/HKDSE/Subject_Information/ict/2025hkdse-e-ict.pdf
3. Python Software Foundation. “csv — CSV File Reading and Writing.” https://docs.python.org/3/library/csv.html
4. Python Software Foundation. “dataclasses — Data Classes.” https://docs.python.org/3/library/dataclasses.html
5. Python Software Foundation. “secrets — Generate Secure Random Numbers.” https://docs.python.org/3/library/secrets.html
6. Python Software Foundation. “collections — Container Datatypes.” https://docs.python.org/3/library/collections.html
7. pytest development team. “pytest Documentation.” https://docs.pytest.org/en/stable/
8. Astral. “Running Commands in Projects.” https://docs.astral.sh/uv/concepts/projects/run/

9.2: Gantt Chart

The following chart expresses the iterative development schedule. Analysis and design overlap because rule discoveries feed back into constraints. Testing begins before feature completion and continues through packaging and documentation.

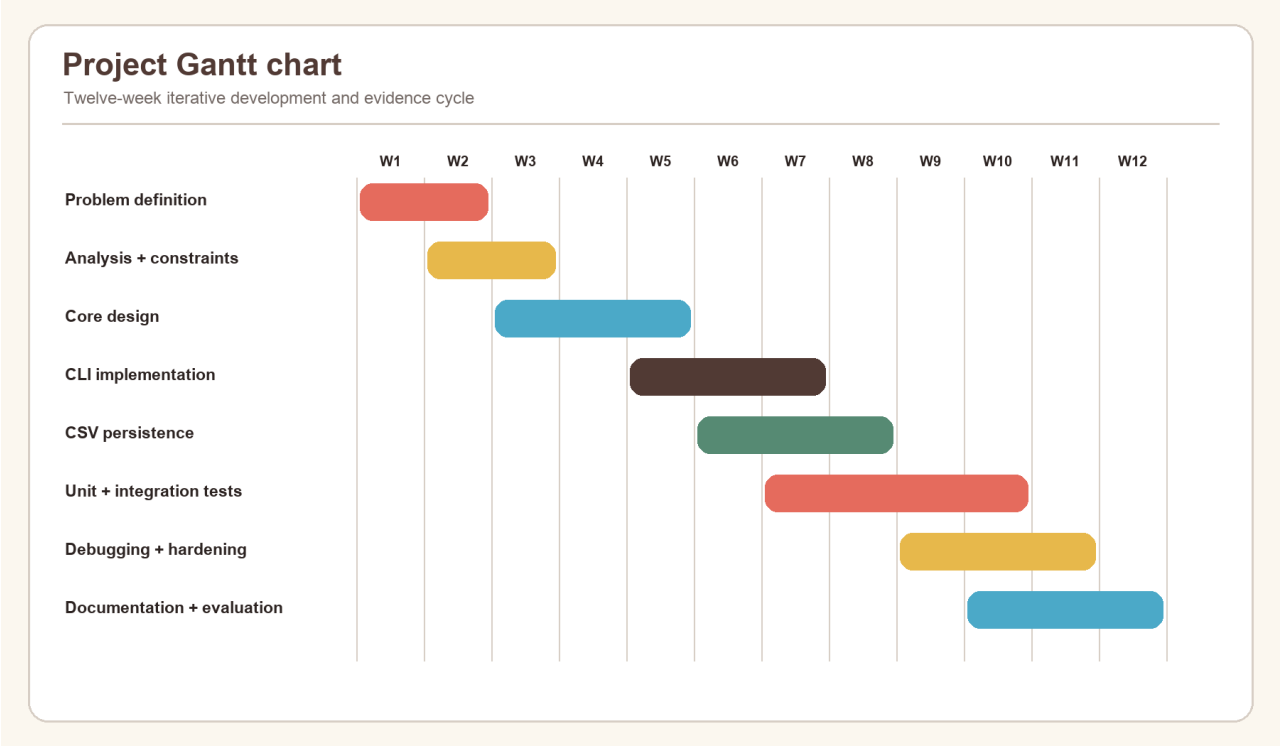

Figure 22. Project schedule from analysis through verification and report completion.

Phase	Main deliverable	Completion evidence
Problem definition	Objectives and minimum expectations	Sections 1–2
Analysis	Parameters, constraints, IPO, modular boundaries	Section 3
Design	State, flow, generation, persistence	Section 4
Implementation	Core, CLI, storage, packaging	Section 5 and source figures
Testing	Unit, integration, system and acceptance evidence	Section 6
Debugging	Seven resolved defect classes and improvements	Section 7
Documentation	README, user manual, final SBA report	Sections 7.4–9